Mastering Diffusion

Week 2 – Generative Modeling I

Introduction to Generative Modeling

Jihwan Hong

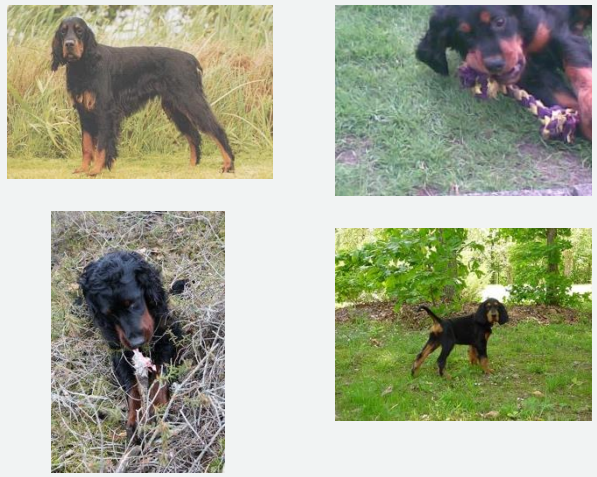
AIDAS Lab
Seoul National University
csjihwanh@gmail.com

AIDAS Lab

Generative Modeling

Generative Models

- Generative model is a model that can **generate new data instances**

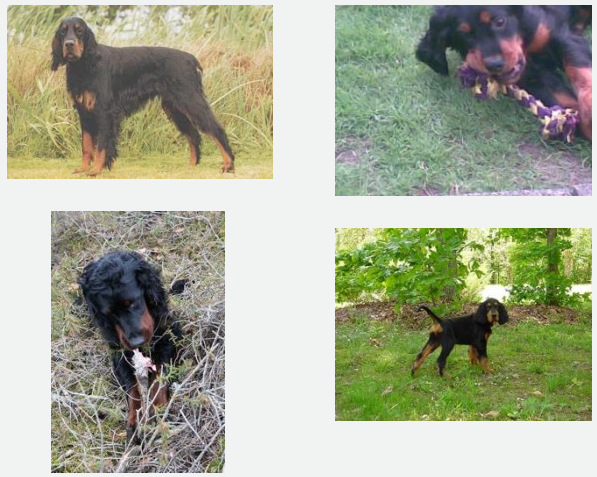


Data X

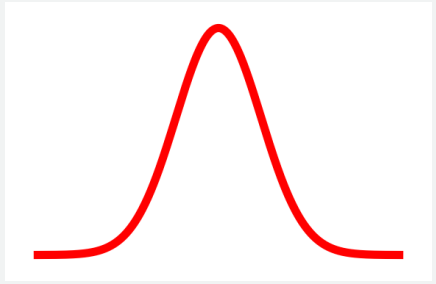
Images from ILSVRC 2012 (Russakovsky et al., IJCV 2015)

Generative Models

- Generative model is a model that can **generate new data instances**
- More precisely, it models the **probability distribution $p(x)$** from data



Learn



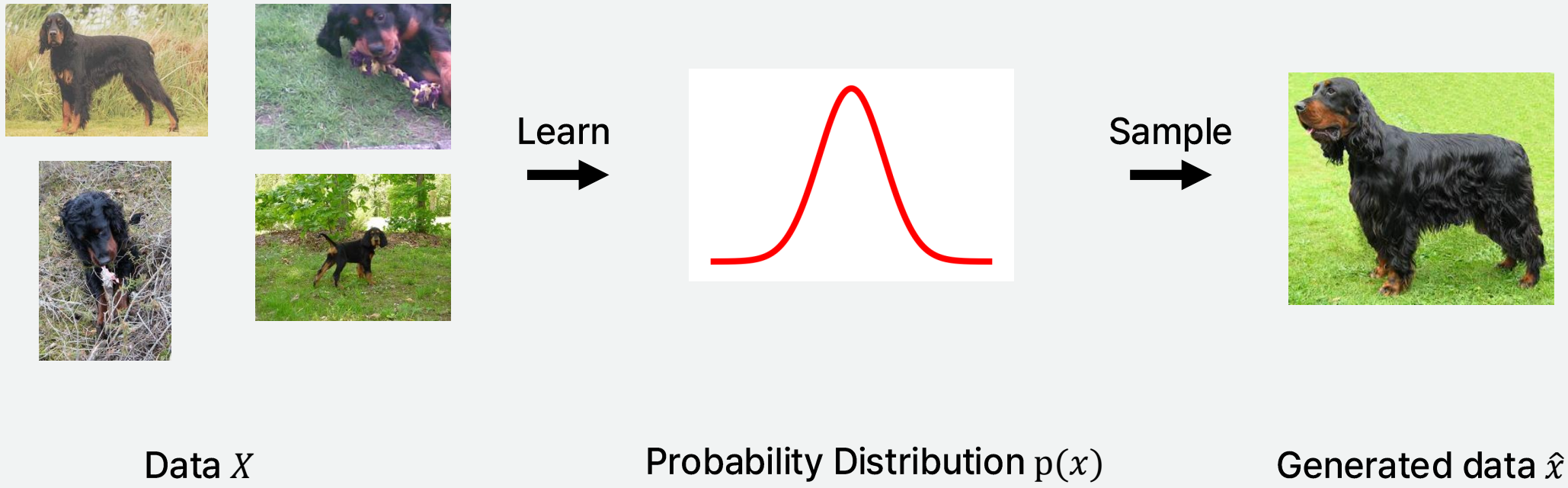
Data X

Probability Distribution $\mathrm{p}(x)$

Images from ILSVRC 2012 (Russakovsky et al., IJCV 2015)

Generative Models

- Generative model is a model that can **generate new data instances**
- More precisely, it models the **probability distribution $p(x)$** from data



Images from ILSVRC 2012 (Russakovsky et al., IJCV 2015)

Discriminative vs. Generative

Discriminative Model
$p(y|x)$

Generative Model
$p(x)$ or $p(x, y)$

Discriminative vs. Generative

Discriminative Model
$p(y|x)$

Generative Model
$p(x)$ or $p(x, y)$

Discriminative models estimate p(y|x) for predefined labels

Discriminative vs. Generative

Discriminative Model
$p(y|x)$

Generative Model
$p(x)$ or $p(x, y)$

Discriminative models estimate p(y|x) for predefined labels



x: image

y: pig

Discriminative vs. Generative

Discriminative Model
$p(y|x)$

Generative Model
$p(x)$ or $p(x, y)$

Discriminative models estimate p(y|x) for predefined labels



Label: pig

Discriminative vs. Generative

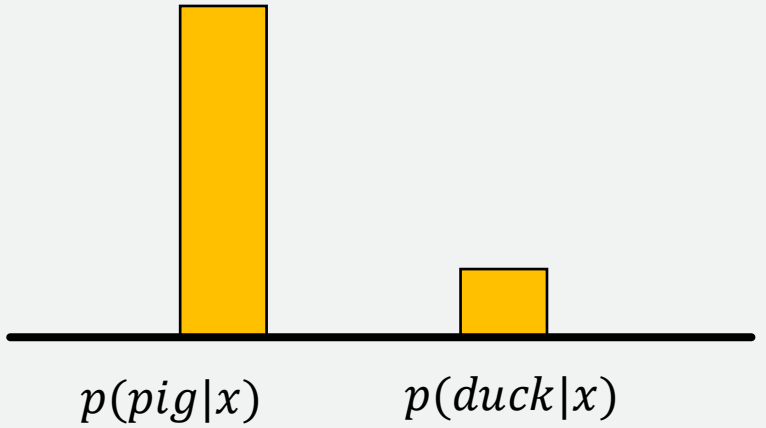
Discriminative Model
$p(y|x)$

Generative Model
$p(x)$ or $p(x, y)$



Label: pig

Classify



Prediction: pig

Discriminative vs. Generative

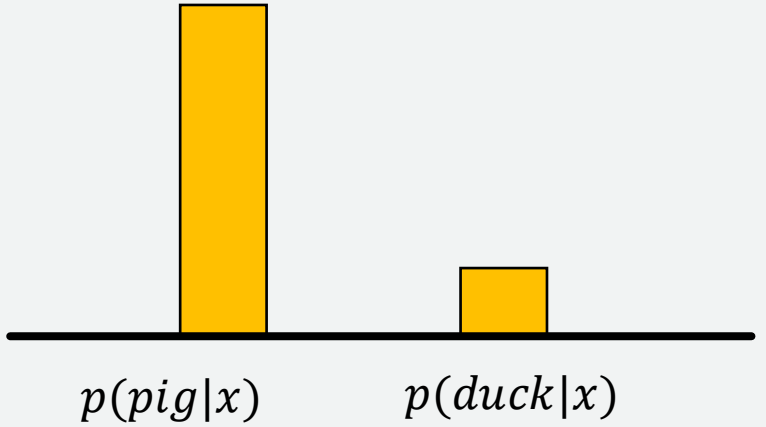
Discriminative Model
$p(y|x)$

Generative Model
$p(x)$ or $p(x, y)$



Label: pig

Classify



Prediction: pig

$$\int p(y|x)dy = 1$$

Discriminative vs. Generative

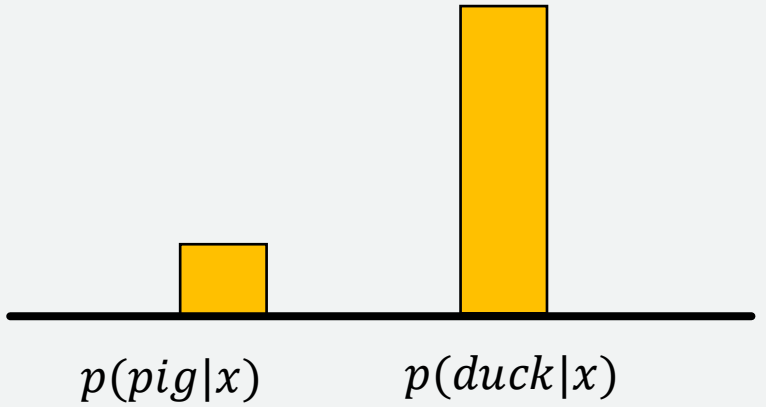
Discriminative Model
$p(y|x)$

Generative Model
$p(x)$ or $p(x, y)$



Label: duck

Classify



Prediction: duck

Discriminative vs. Generative

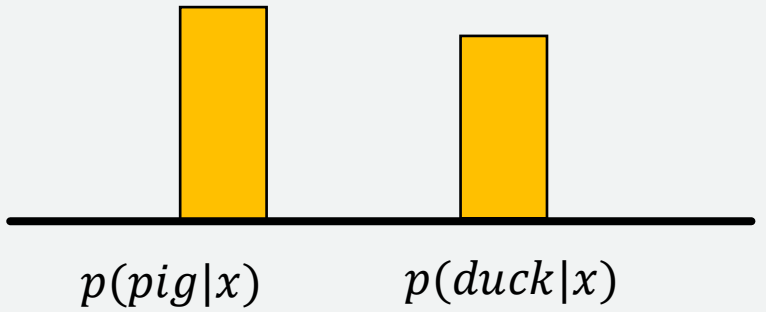
Discriminative Model
$p(y|x)$

Generative Model
$p(x)$ or $p(x, y)$



Label: dog

Classify



Prediction: pig

Images from ILSVRC 2012 (Russakovsky et al., IJCV 2015)

Discriminative vs. Generative

Discriminative Model
$p(y|x)$

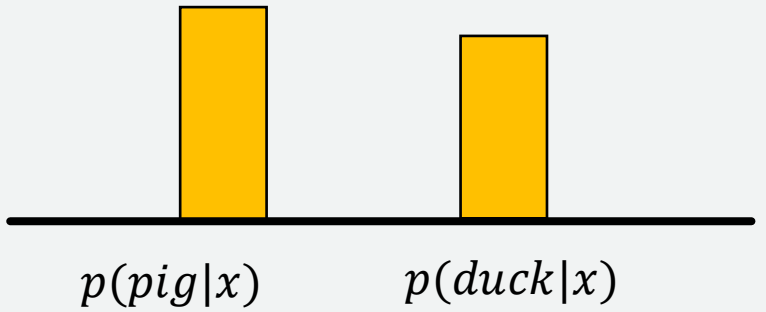
Generative Model
$p(x)$ or $p(x, y)$



Label: dog

No way to compute p(y|x) for undefined labels

Classify



Prediction: pig

Images from ILSVRC 2012 (Russakovsky et al., IJCV 2015)

Discriminative vs. Generative

Discriminative Model
$p(y|x)$

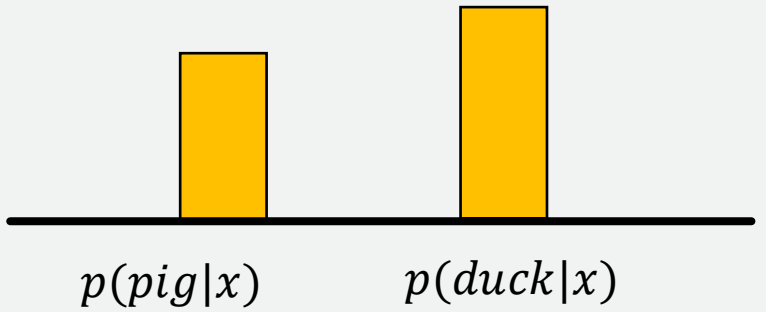
Generative Model
$p(x)$ or $p(x, y)$



Label: ??

No way to compute p(y|x) for undefined labels

Classify



Prediction: duck

Discriminative vs. Generative

Discriminative Model
$p(y|x)$

Generative Model
$p(x)$ or $p(x, y)$

Generative models estimate how likely an image is

Discriminative vs. Generative

Discriminative Model
$p(y|x)$

Generative Model
$p(x)$ or $p(x, y)$

Generative models estimate how likely an image is

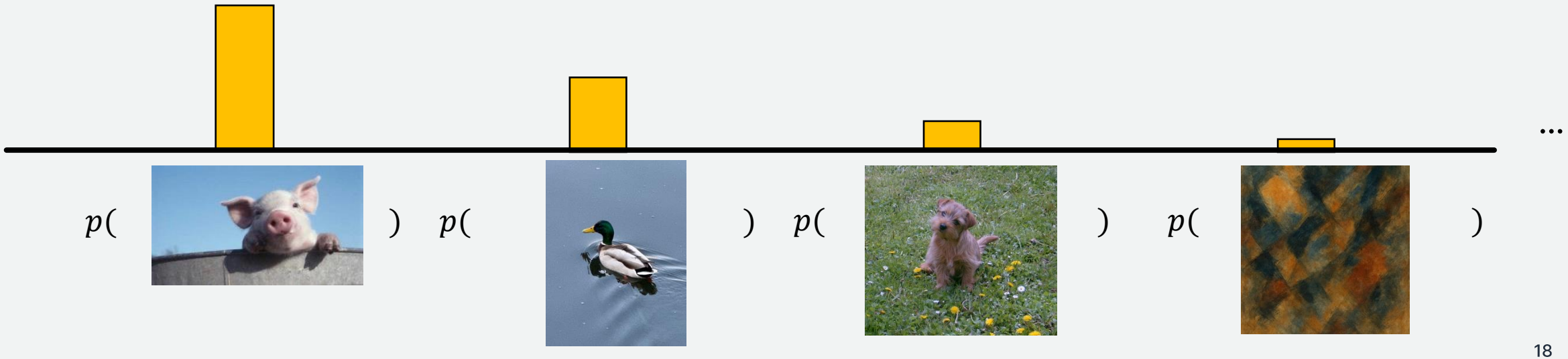


Discriminative vs. Generative

Discriminative Model
$p(y|x)$

Generative Model
$p(x)$ or $p(x, y)$

Generative models estimate how likely an image is



Images from ILSVRC 2012 (Russakovsky et al., IJCV 2015)

Discriminative vs. Generative

Discriminative Model
$p(y|x)$

Generative Model
$p(x)$ or $p(x, y)$

Or, generative models estimate how likely x and y occur together

Discriminative vs. Generative

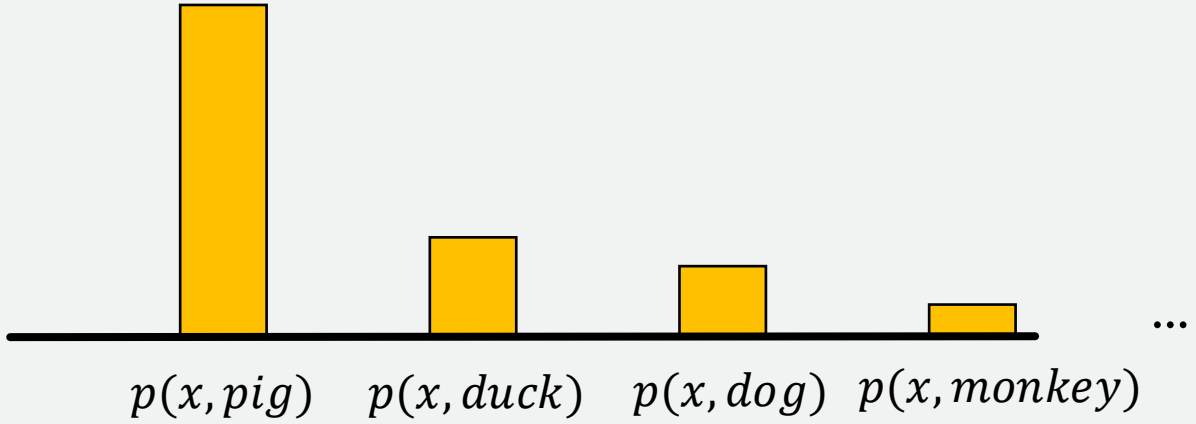
Discriminative Model
$p(y|x)$

Generative Model
$p(x)$ or $p(x, y)$

Or, generative models estimate how likely x and y occur together



Label: pig



Discriminative vs. Generative

Discriminative Model
$p(y|x)$

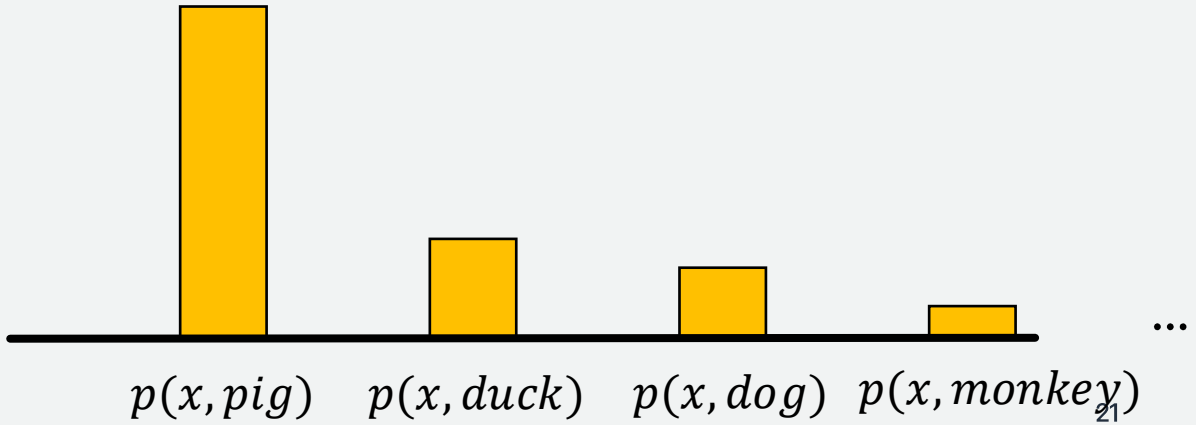
Generative Model
$p(x)$ or $p(x, y)$

Or, generative models estimate how likely x and y occur together

We can build a discriminative model from this



Label: pig



Discriminative vs. Generative

Discriminative Model
$p(y|x)$

Generative Model
$p(x)$ or $p(x, y)$

Or, generative models estimate how likely x and y occur together

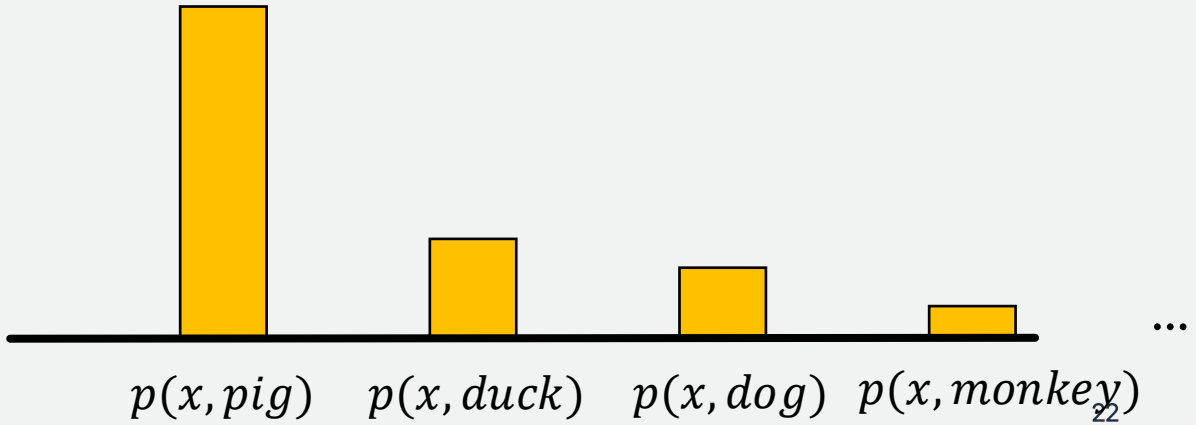
We can build a discriminative model from this

Recap: Bayes' Rule

$$p(y|x) = \frac{p(y)p(x|y)}{p(x)} = \frac{p(x, y)}{p(x)}$$



Label: pig



Discriminative vs. Generative

Discriminative Model
$p(y|x)$

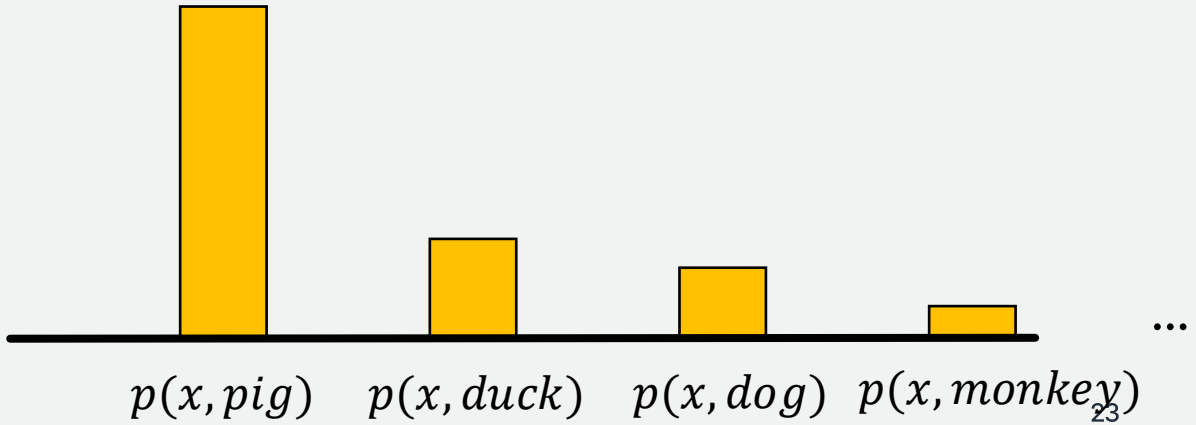
Generative Model
$p(x)$ or $p(x, y)$

Or, generative models estimate how likely x and y occur together

We can build a discriminative model from this $p(pig|x) = \frac{p(x, pig)}{p(x)}$



Label: pig



Discriminative vs. Generative

Discriminative Model
$p(y|x)$

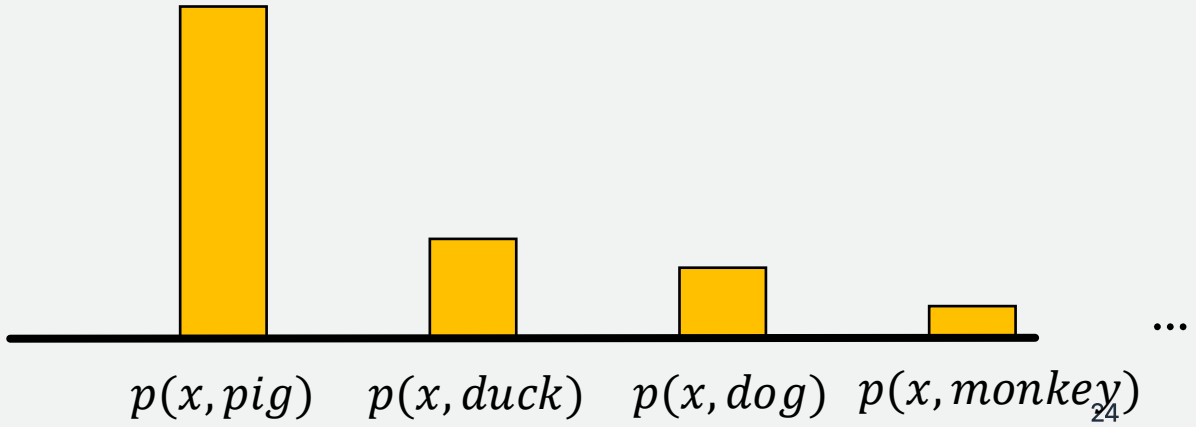
Generative Model
$p(x)$ or $p(x, y)$

Or, generative models estimate how likely x and y occur together

We can build a discriminative model from this $p(pig|x) = \frac{p(x, pig)}{p(x)}$ $p(duck|x) = \frac{p(x, duck)}{p(x)}$



Label: pig



Deep Generative Models

Deep Generative Models

Deep Generative Models

Explicitness: define an explicit form of p(x)?

Deep Generative Models ———

Deep Generative Models

Deep Generative Models ———

Explicitness: define an **explicit form of p(x)**?

Tractability: can **p(x) be computed** exactly and efficiently?

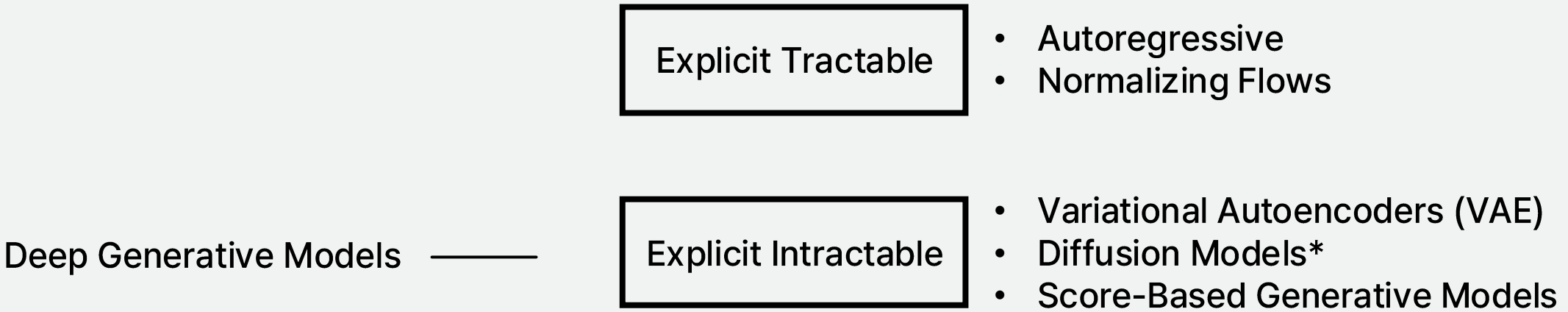
Deep Generative Models

Explicit Tractable

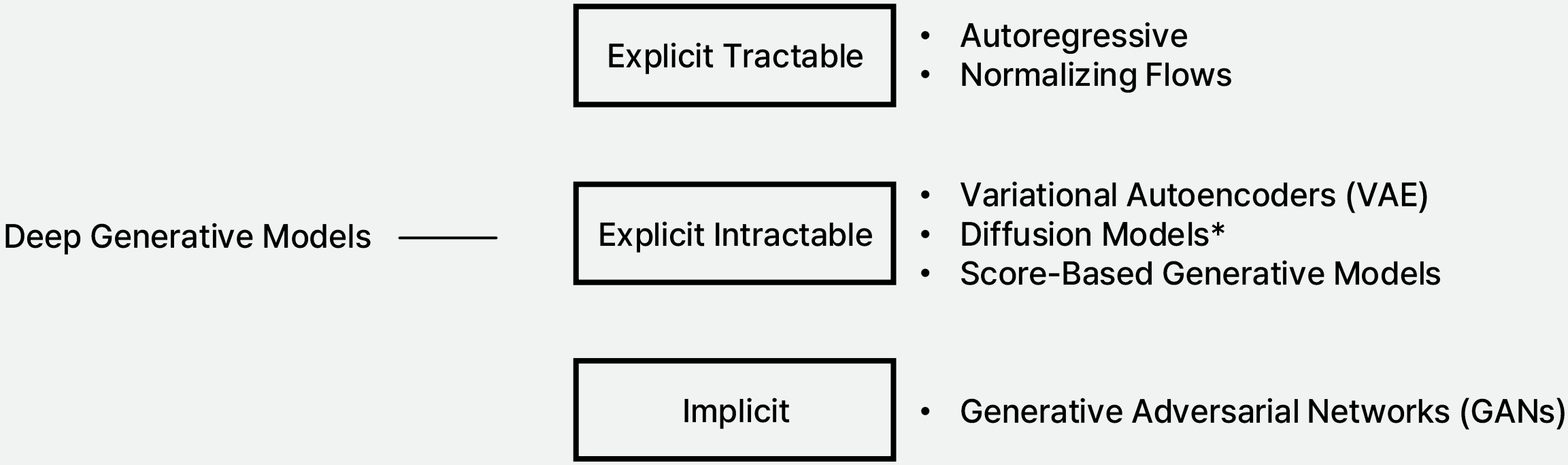
- Autoregressive
- Normalizing Flows

Deep Generative Models

Deep Generative Models



Deep Generative Models



Autoregressive Models

Maximum Likelihood Estimation

- Our objective: $p(x) = f(x, W)$

Maximum Likelihood Estimation

- Our objective: $p(x) = f(x, W)$
- Given dataset $x_1, x_2, \ldots x_n$ train the model by solving:

Maximum Likelihood Estimation

- Our objective: $p(x) = f(x, W)$
- Given dataset $x_1, x_2, \ldots x_n$ train the model by solving:

$$W^* = \arg\max_W \prod_i p(x_i)$$

Maximum Likelihood Estimation

- Our objective: $p(x) = f(x, W)$
- Given dataset $x_1, x_2, \ldots x_n$ train the model by solving:

$$W^* = \arg\max_W \prod_i p(x_i)$$

$$= \arg\max_W \sum_i \log p(x_i)$$

Maximum Likelihood Estimation

- Our objective: $p(x) = f(x, W)$
- Given dataset $x_1, x_2, \ldots x_n$ train the model by solving:

$$W^* = \arg\max_W \prod_i p(x_i)$$

$$= \arg\max_W \sum_i \log p(x_i)$$

$$= \arg\max_W \sum_i \log f(x_i, W)$$

Autoregressive Models

- Assume x is a sequence: $x = (x_1, x_2, \ldots, x_T)$

Autoregressive Models

- Assume x is a sequence: $x = (x_1, x_2, \dots, x_T)$
- Then:

$$p(x) = p(x_1, x_2, \dots x_T)$$

Autoregressive Models

- Assume x is a sequence: $x = (x_1, x_2, \dots, x_T)$
- Then:

$$p(x) = p(x_1, x_2, \dots x_T)$$

$$= p(x_1)p(x_2|x_1)p(x_3|x_1, x_2) \cdots$$

Autoregressive Models

- Assume x is a sequence: $x = (x_1, x_2, \dots, x_T)$
- Then:

$$p(x) = p(x_1, x_2, \dots x_T)$$

$$= p(x_1)p(x_2|x_1)p(x_3|x_1, x_2) \cdots$$

$$= \prod_{t=1}^{T} p(x_t|x_1, \dots, x_{t-1})$$

PixelRNN

- Image $\mathbf{x}$: $n \times n$ pixels

van Den Oord et al., “Pixel recurrent neural networks”, ICML 2016.

PixelRNN

- Image $\mathbf{x}$: $n \times n$ pixels
- Make them into n^2 sequence

van Den Oord et al., “Pixel recurrent neural networks”, ICML 2016.

PixelRNN

- Image $\mathbf{x}$: $n \times n$ pixels
- Make them into n^2 sequence
- Then:

$$p(\mathbf{x}) = p(x_1, x_2, \ldots x_{n^2})$$

van Den Oord et al., “Pixel recurrent neural networks”, ICML 2016.

PixelRNN

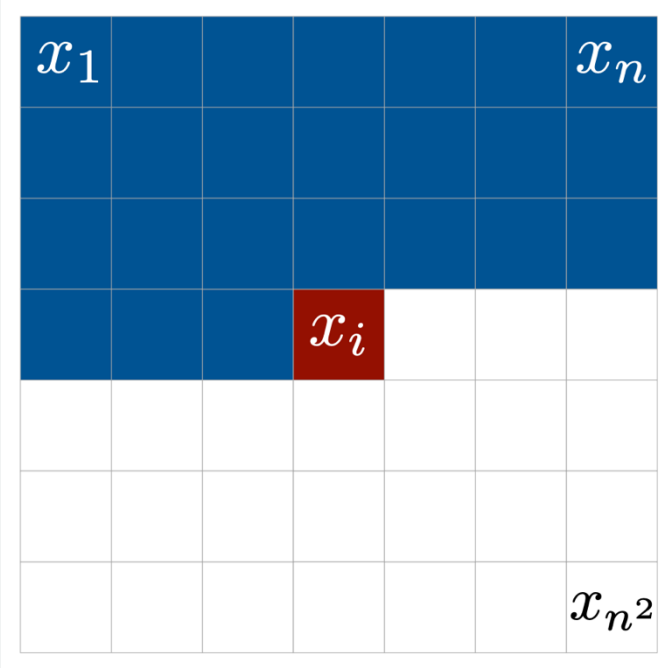
- Image $\mathbf{x}$: $n \times n$ pixels
- Make them into n^2 sequence
- Then:

$$p(\mathbf{x}) = p(x_1, x_2, \ldots x_{n^2})$$

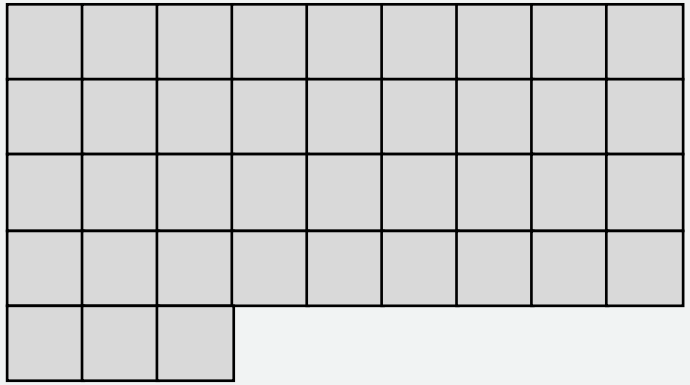
$$= \prod_{i=1}^{n^2} p(x_i | x_1, \ldots, x_{i-1})$$

van Den Oord et al., "Pixel recurrent neural networks", ICML 2016.

PixelRNN

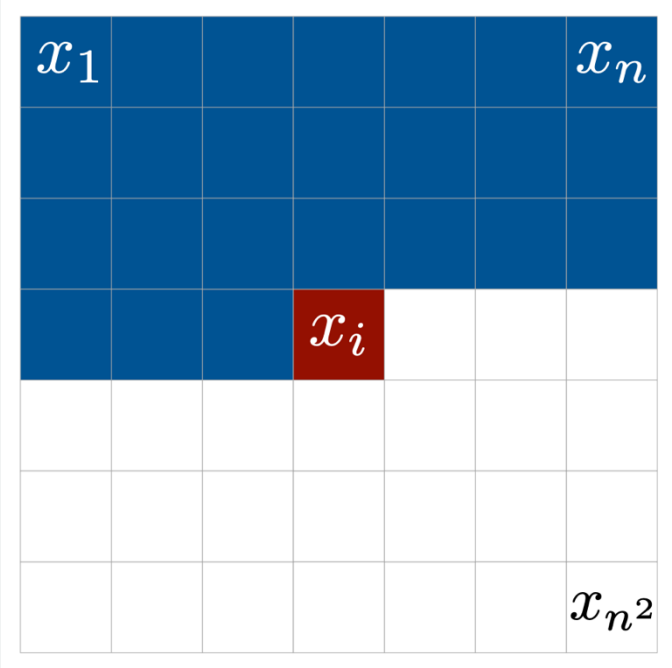


Context

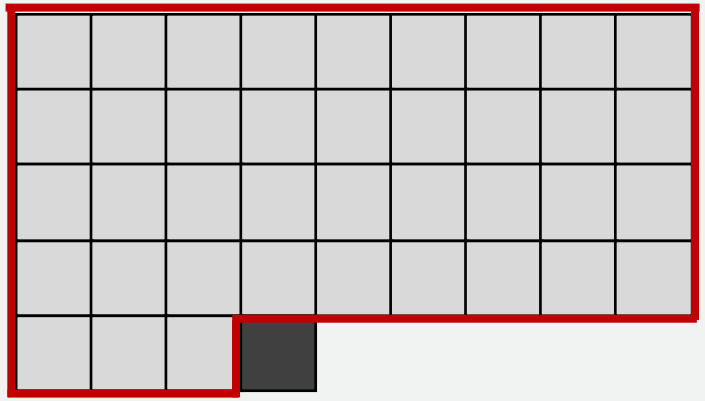


van Den Oord et al., “Pixel recurrent neural networks”, ICML 2016.

PixelRNN

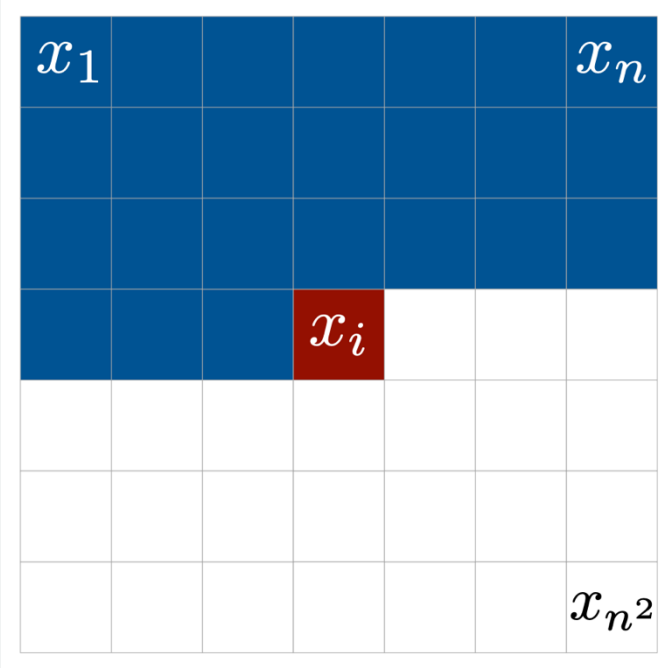


Context

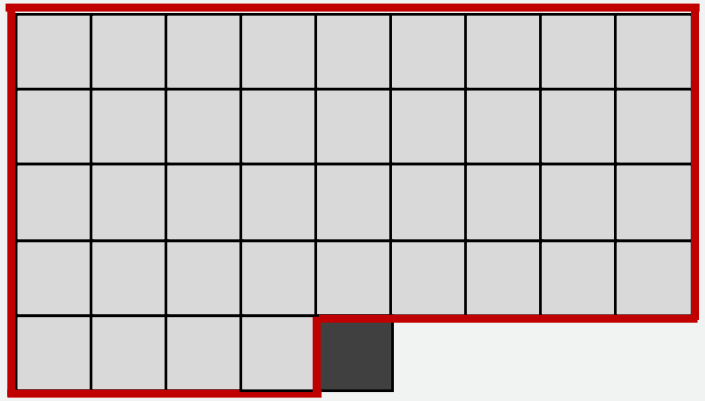


van Den Oord et al., “Pixel recurrent neural networks”, ICML 2016.

PixelRNN

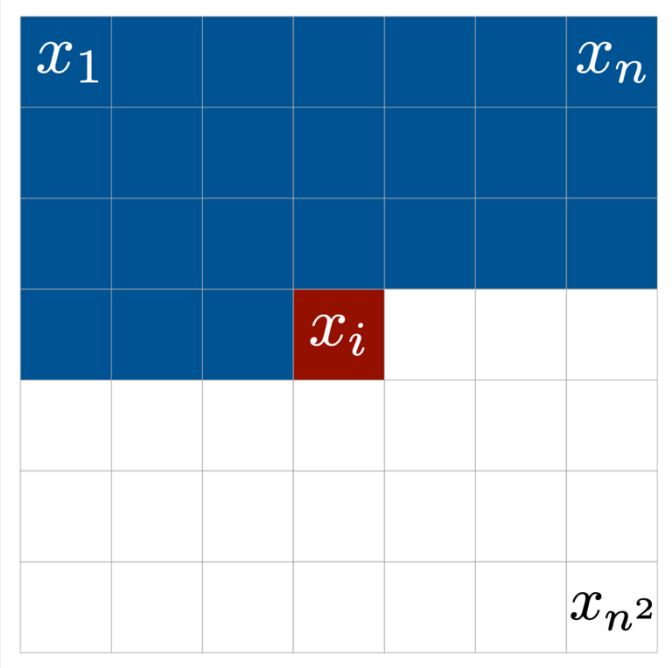


Context

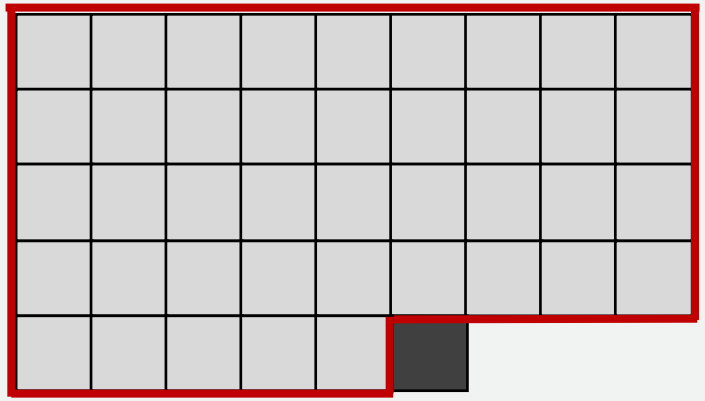


van Den Oord et al., "Pixel recurrent neural networks", ICML 2016.

PixelRNN

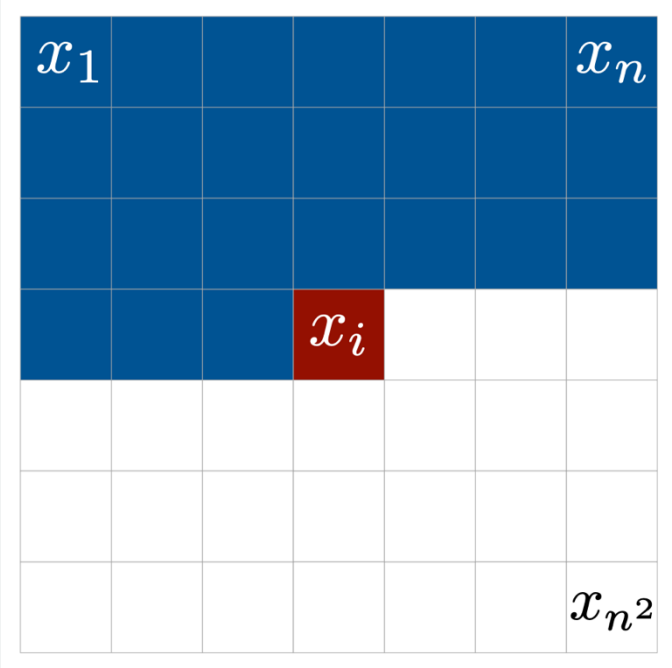


Context

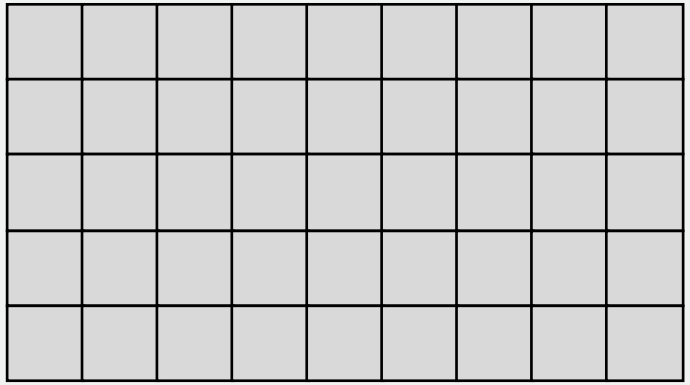


van Den Oord et al., “Pixel recurrent neural networks”, ICML 2016.

PixelRNN

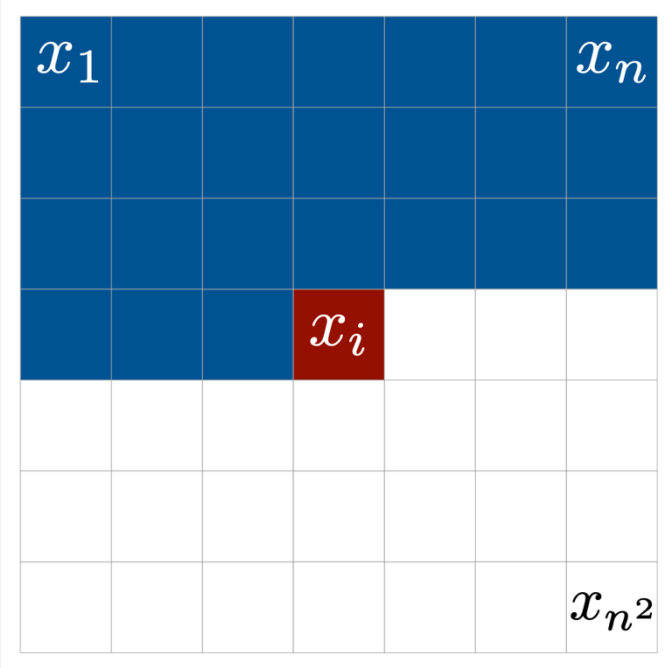


Context

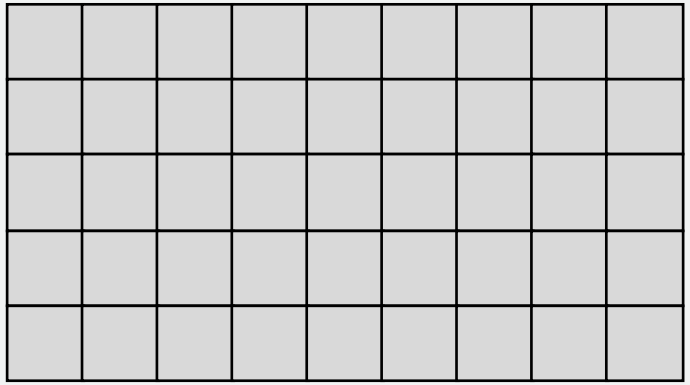


van Den Oord et al., “Pixel recurrent neural networks”, ICML 2016.

PixelRNN



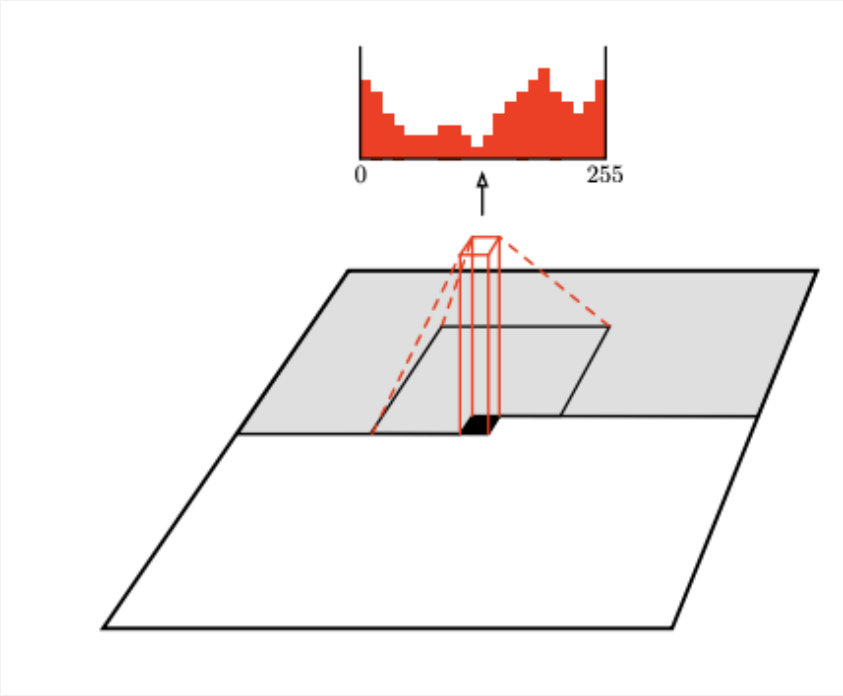
Context

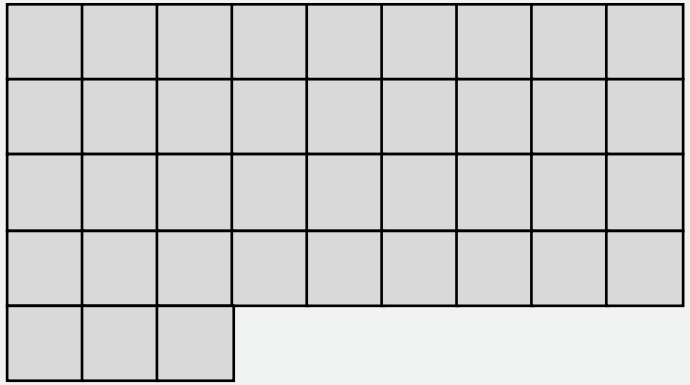


Sequential generation is extremely slow;
generating 256×256 image requires 65K steps

van Den Oord et al., "Pixel recurrent neural networks", ICML 2016.

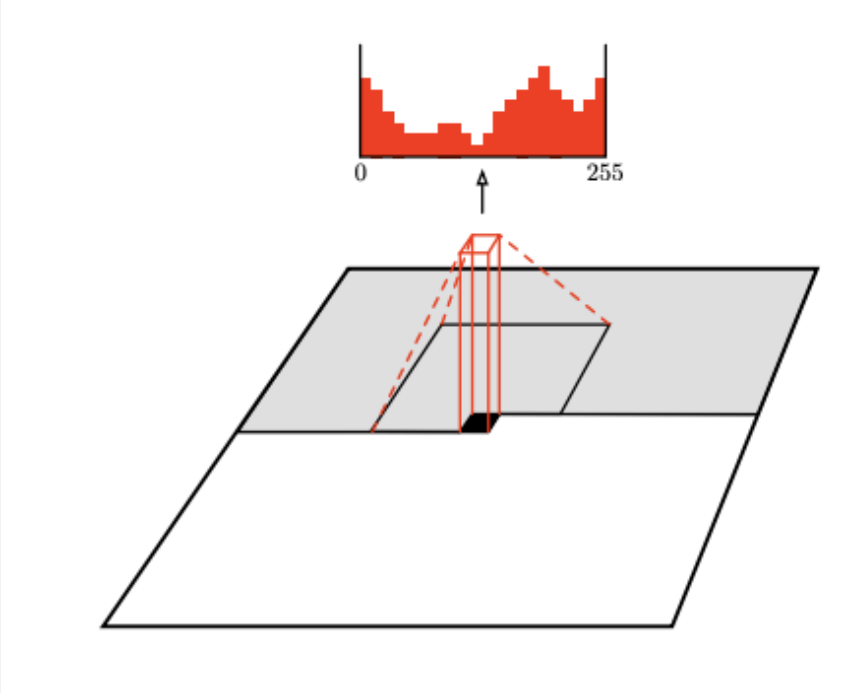
PixelCNN

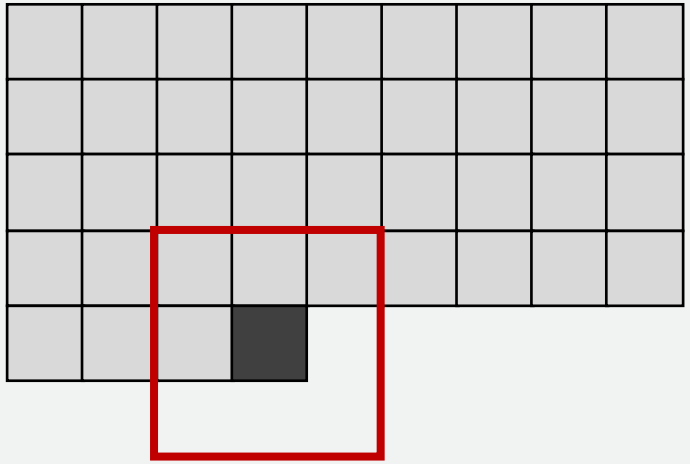




van Den Oord et al., “Pixel recurrent neural networks”, ICML 2016.

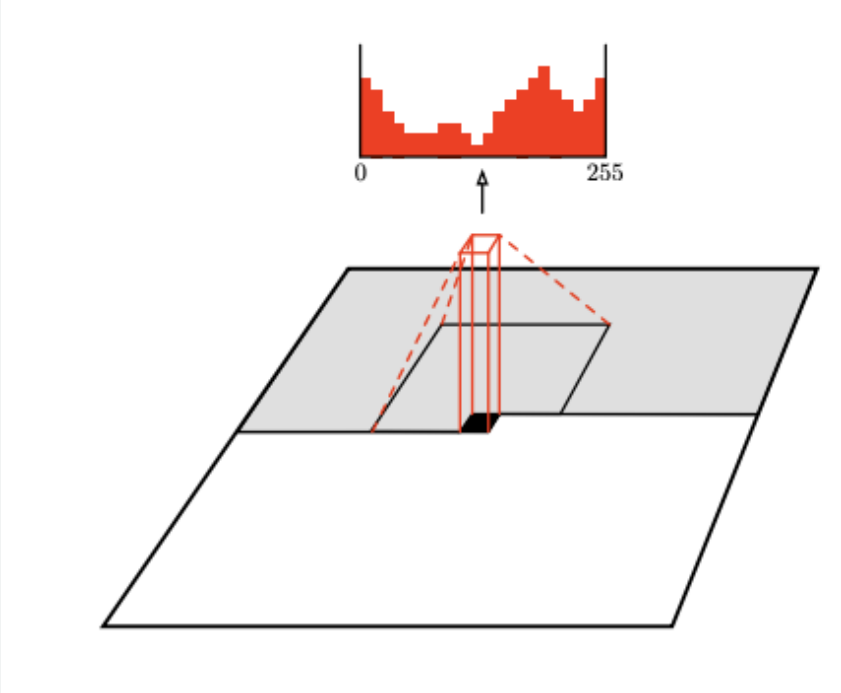
PixelCNN

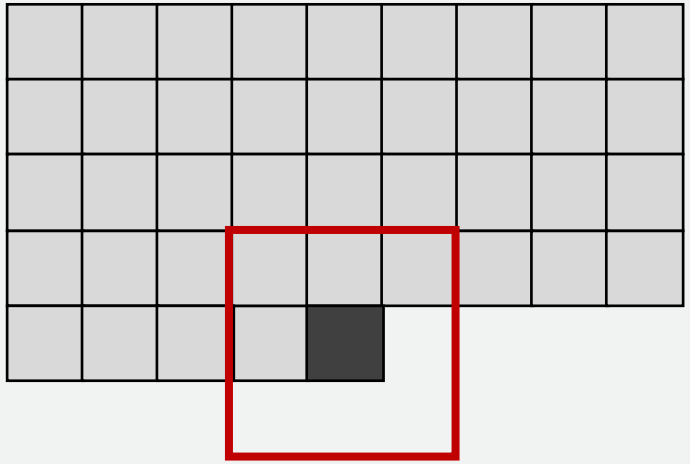




van Den Oord et al., “Pixel recurrent neural networks”, ICML 2016.

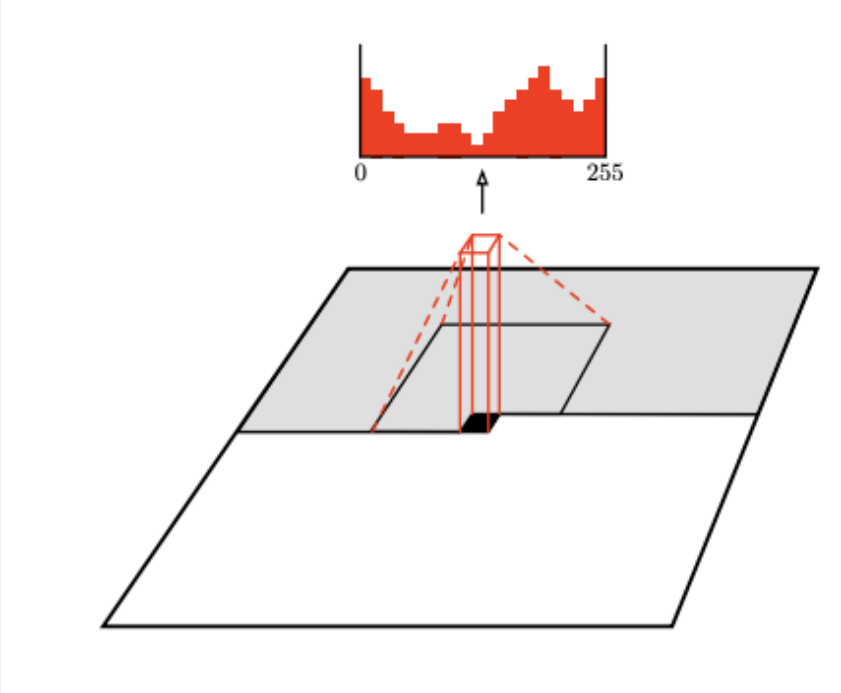
PixelCNN

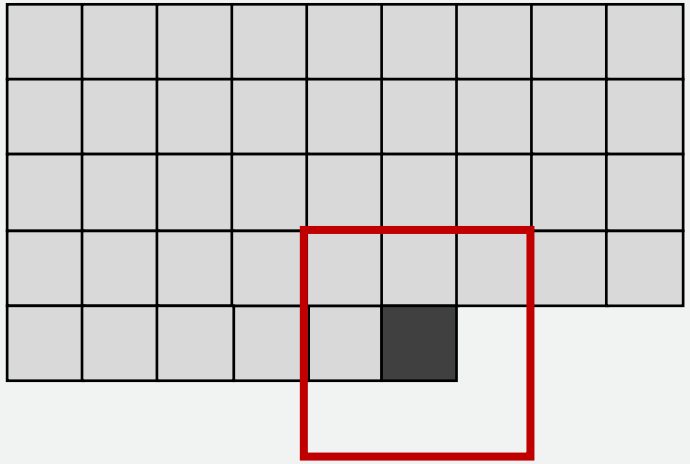




van Den Oord et al., "Pixel recurrent neural networks", ICML 2016.

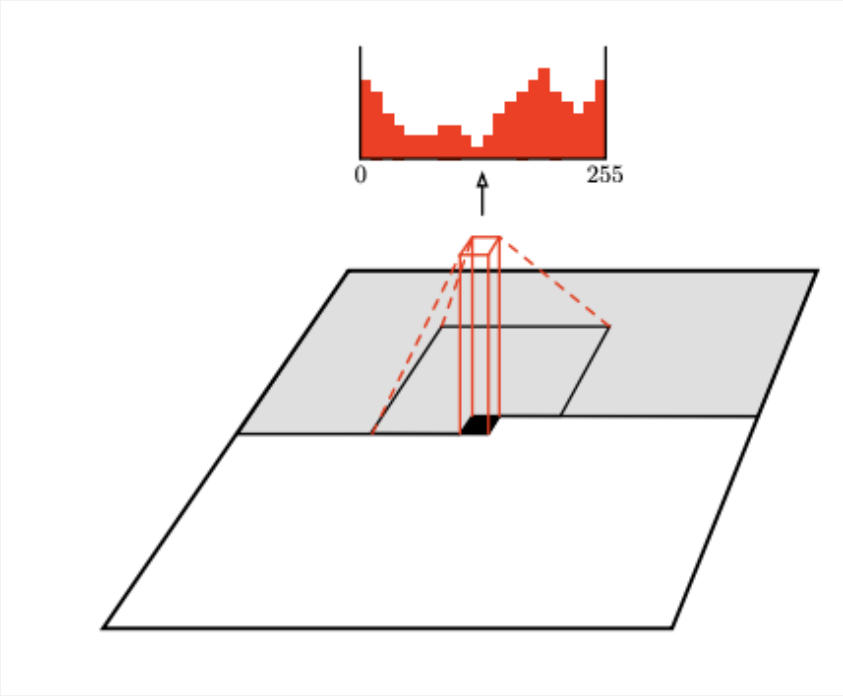
PixelCNN

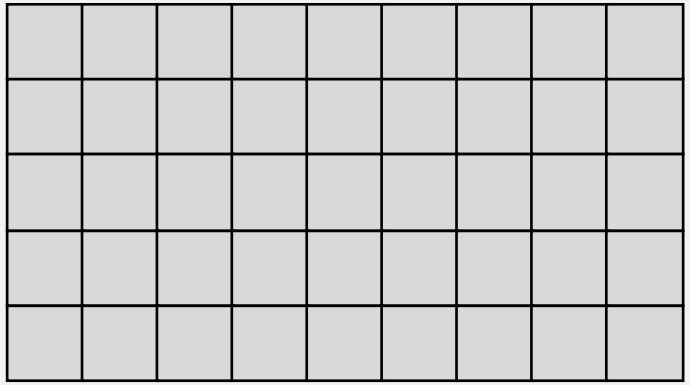




van Den Oord et al., “Pixel recurrent neural networks”, ICML 2016.

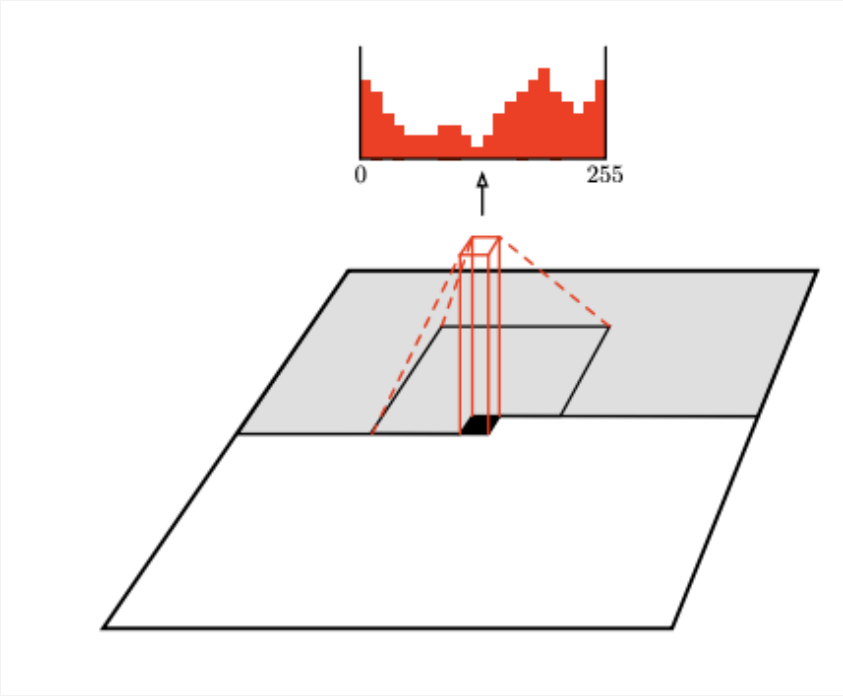
PixelCNN

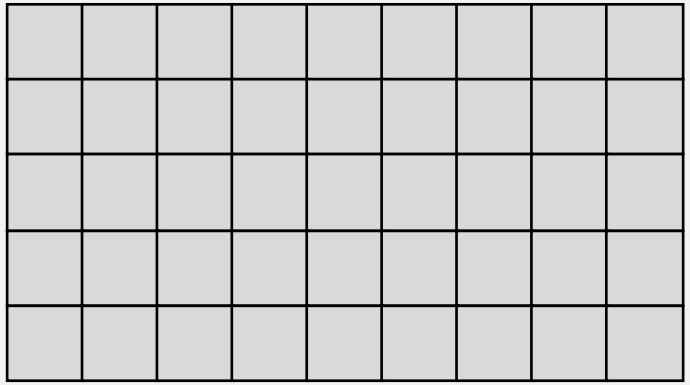




van Den Oord et al., “Pixel recurrent neural networks”, ICML 2016.

PixelCNN

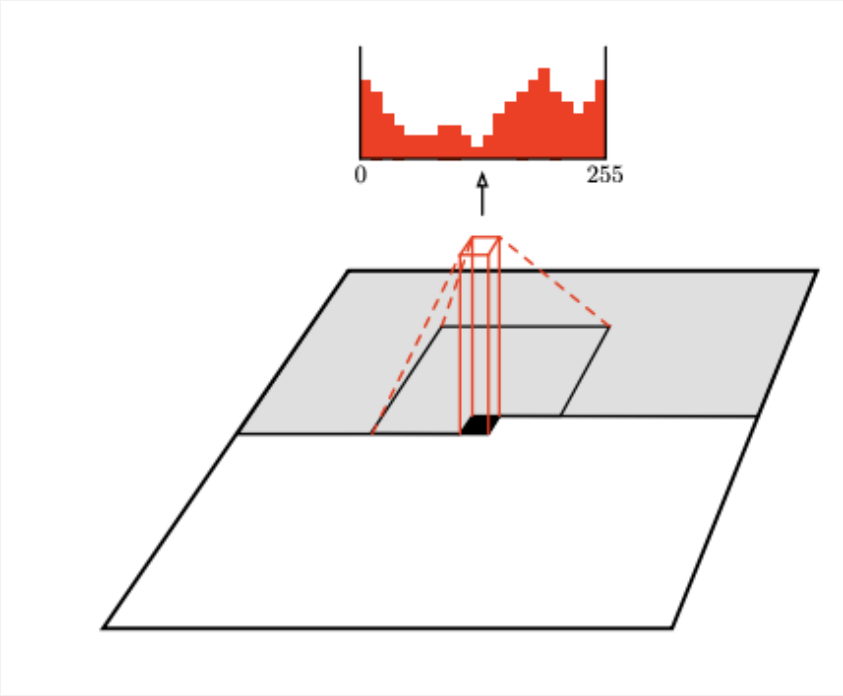


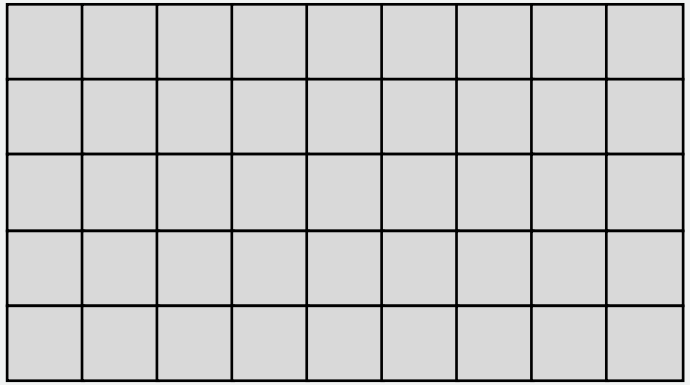


Slightly faster due to parallelizable generation,

van Den Oord et al., “Pixel recurrent neural networks”, ICML 2016.

PixelCNN





Slightly faster due to parallelizable generation, however, still very slow

van Den Oord et al., "Pixel recurrent neural networks", ICML 2016.

PixelRNN Generated Samples



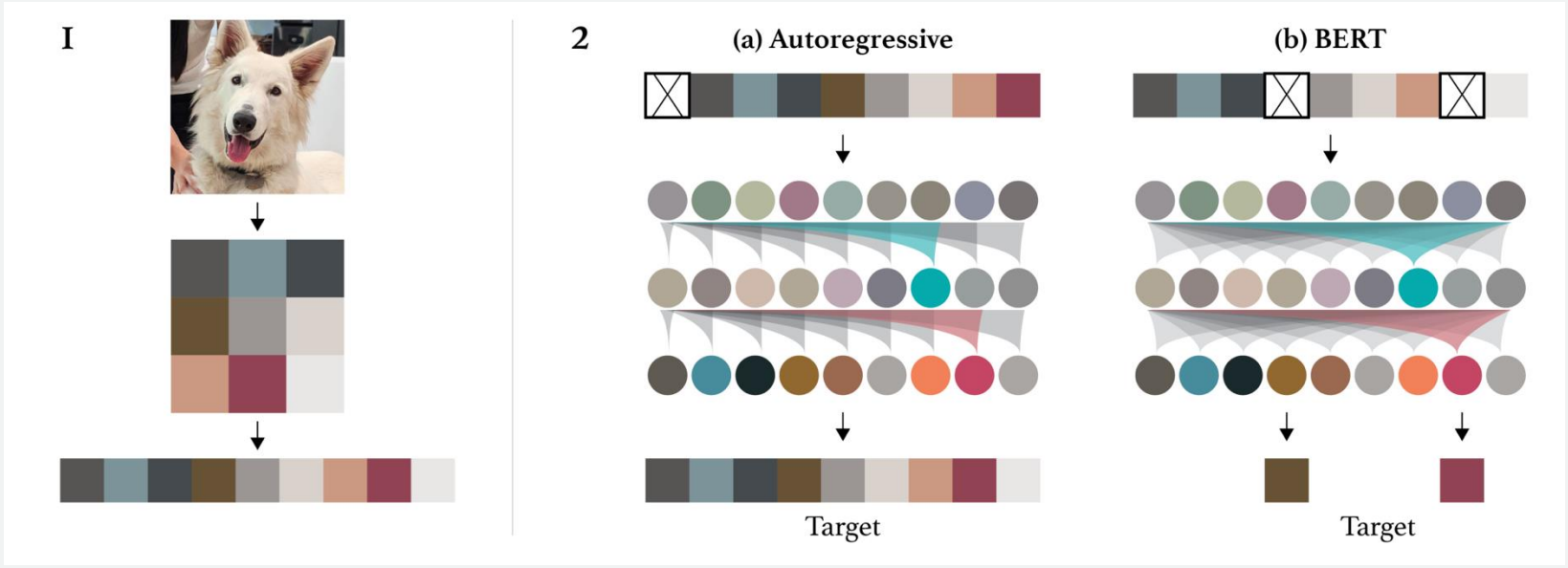
van Den Oord et al., “Pixel recurrent neural networks”, ICML 2016.

PixelCNN Generated Samples



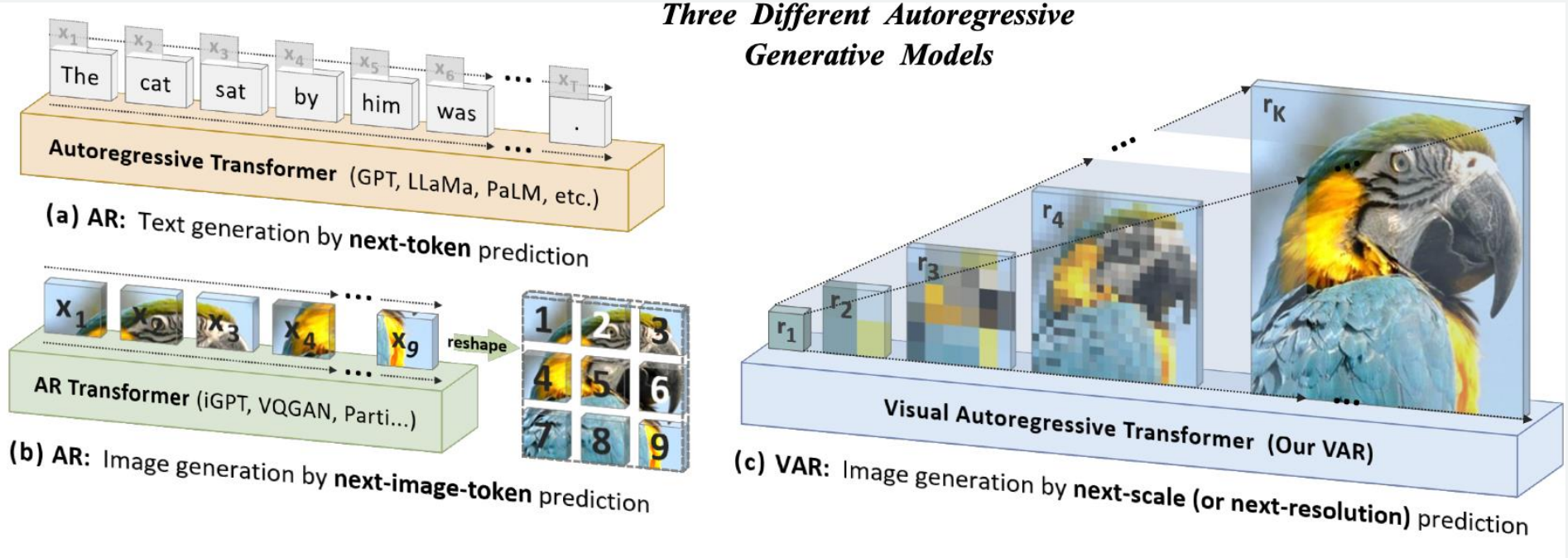
van Den Oord et al., “Pixel recurrent neural networks”, ICML 2016.

Transformer-based Image Generation



Chen et al., "Generative Pretraining from Pixels", ICML 2020.

Transformer-based Image Generation



Tian et al., "Visual Autoregressive Modeling: Scalable Image Generation via Next-Scale Prediction.", NeurIPS 2024.

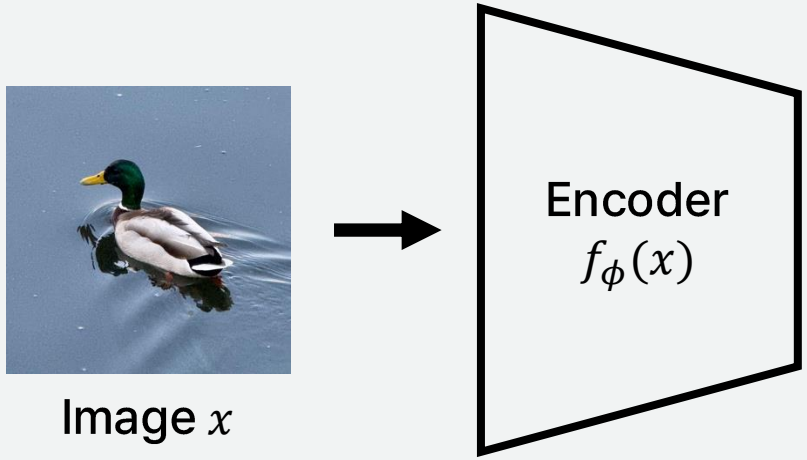
Variational Autoencoder (VAE)

Autoencoder

- Goal: Learn latent representation z without the label

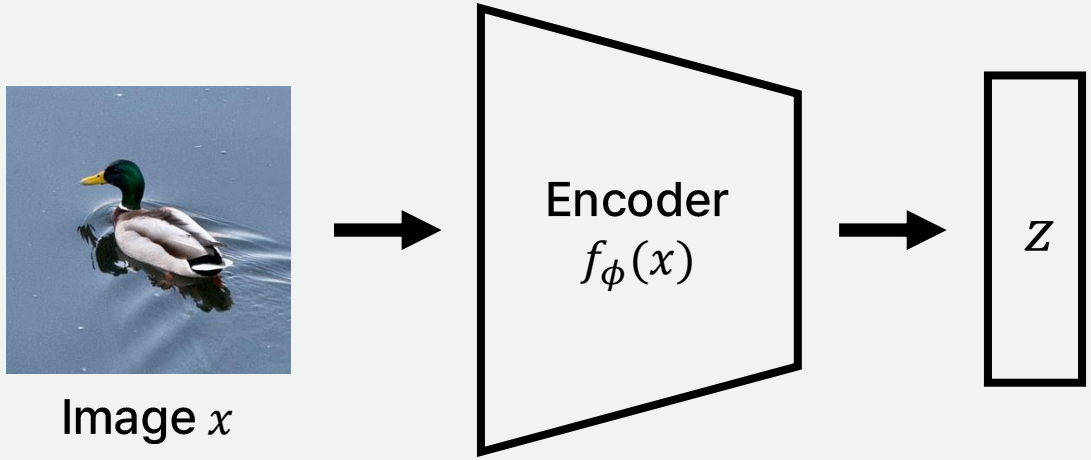
Autoencoder

- Goal: Learn latent representation z without the label



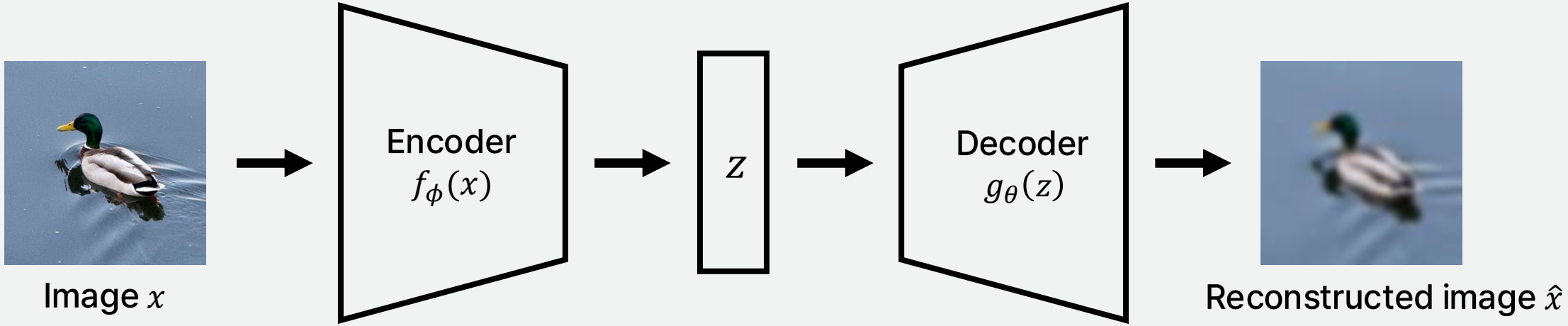
Autoencoder

- Goal: Learn latent representation z without the label



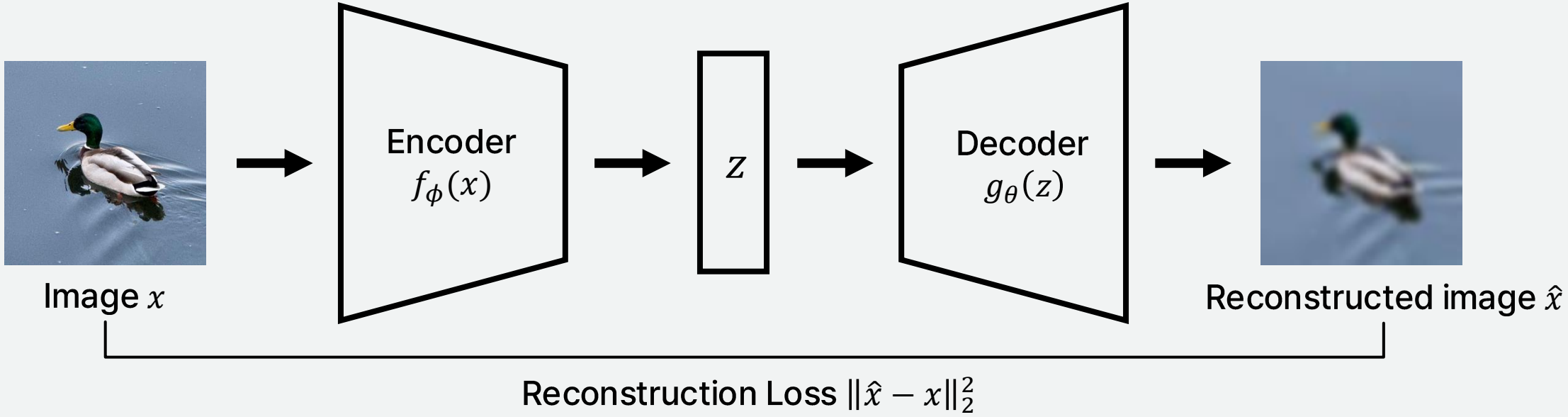
Autoencoder

- Goal: Learn latent representation z without the label



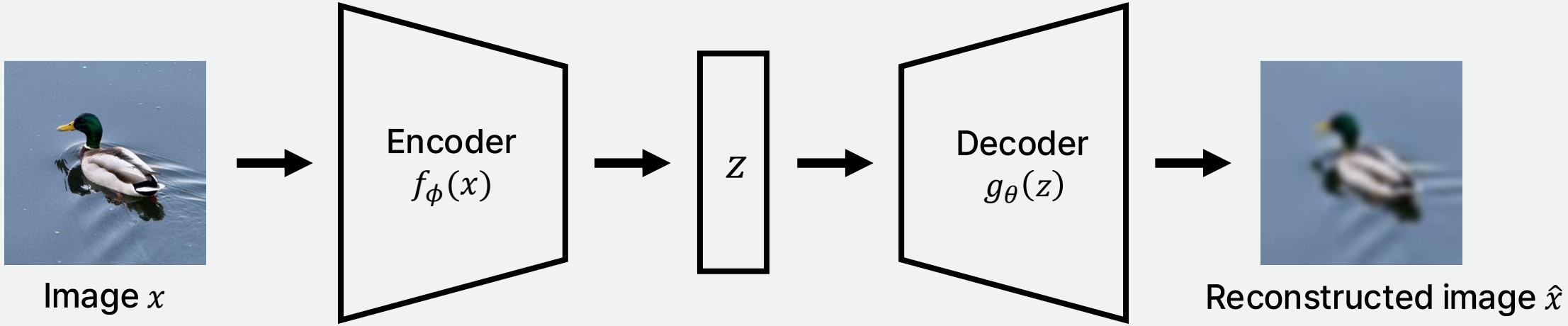
Autoencoder

- Goal: Learn latent representation z without the label



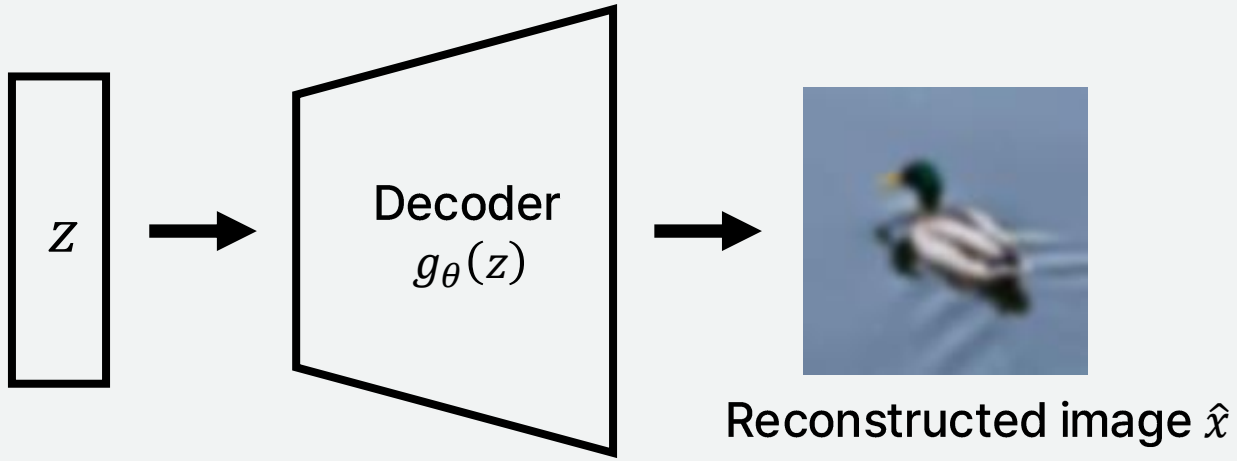
Autoencoder

- How can we generate new data from this?



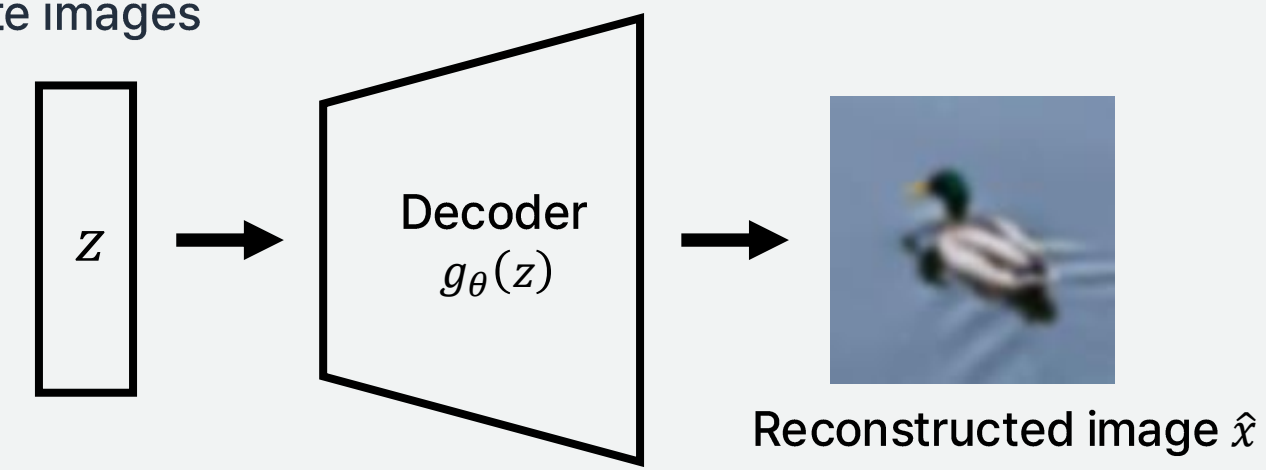
Autoencoder

- How can we generate new data from this?



Autoencoder

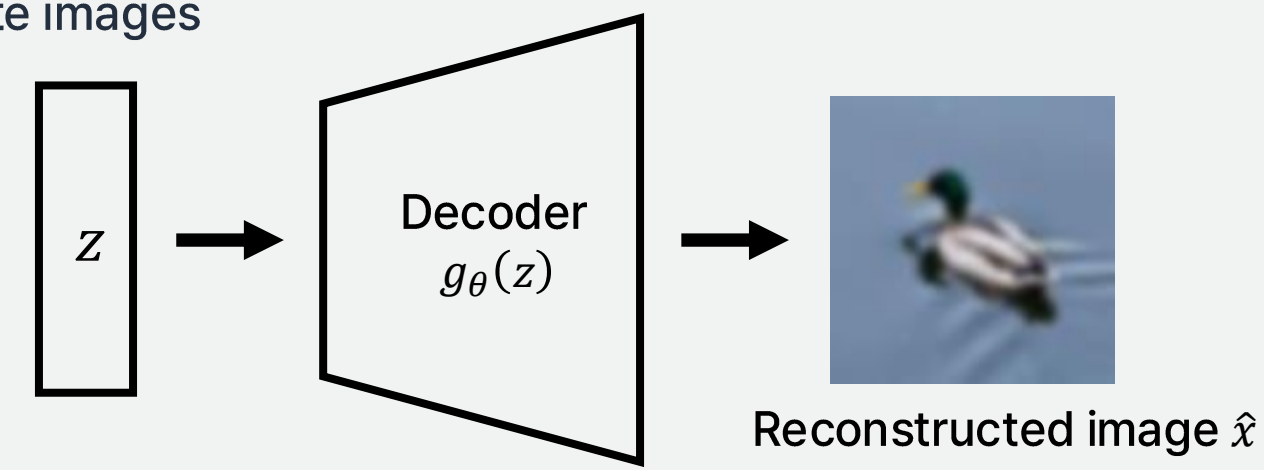
- How can we generate new data from this?
  - Generate a new z to use decoder to generate images



Autoencoder

- How can we generate new data from this?
  - Generate a new z to use decoder to generate images

However, we do not know
the probability distribution of z

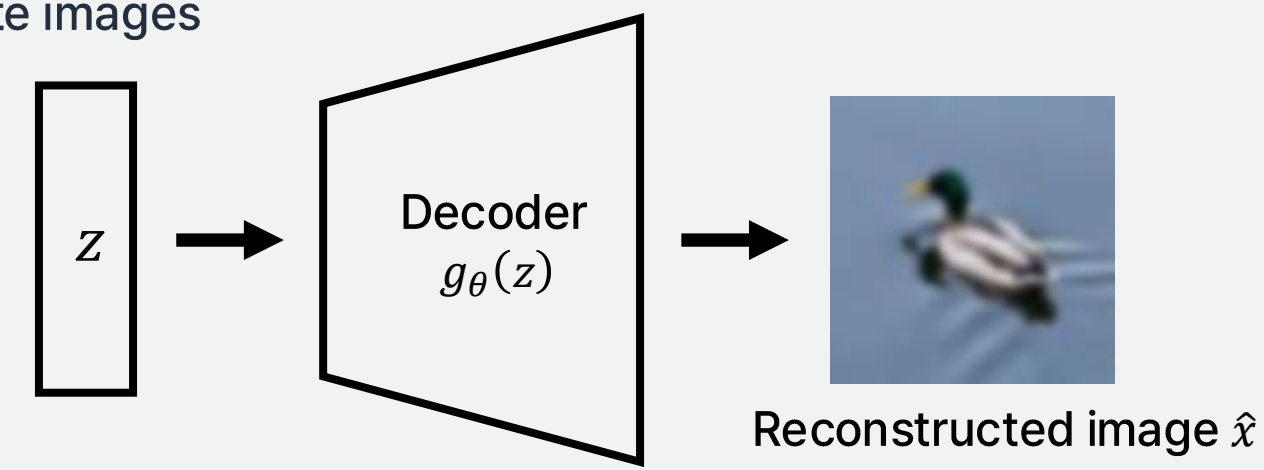


Autoencoder

- How can we generate new data from this?
  - Generate a new z to use decoder to generate images

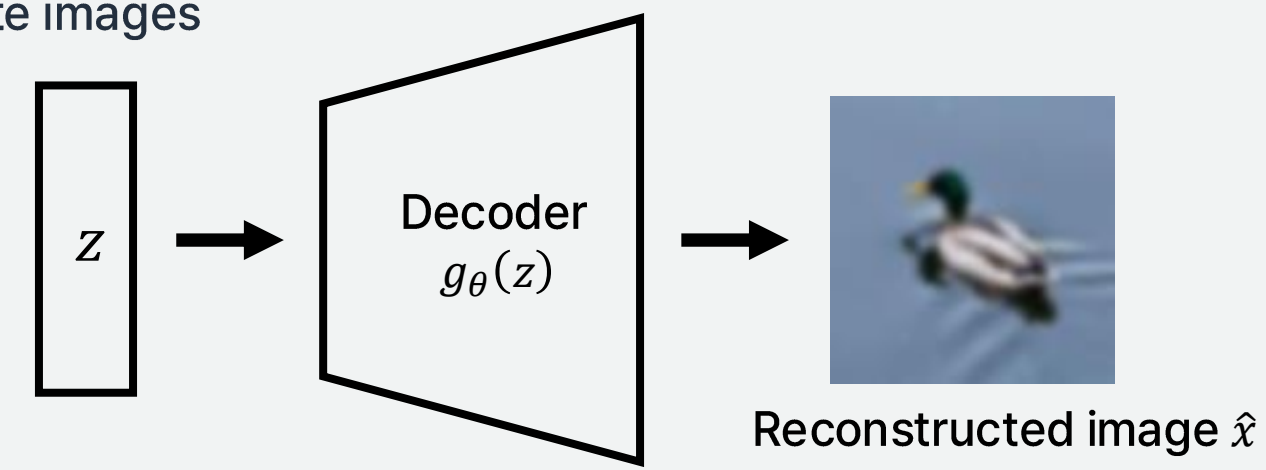
However, we do not know
the probability distribution of z

Can we make the P(z) nicer?



Autoencoder

- How can we generate new data from this?
  - Generate a new z to use decoder to generate images

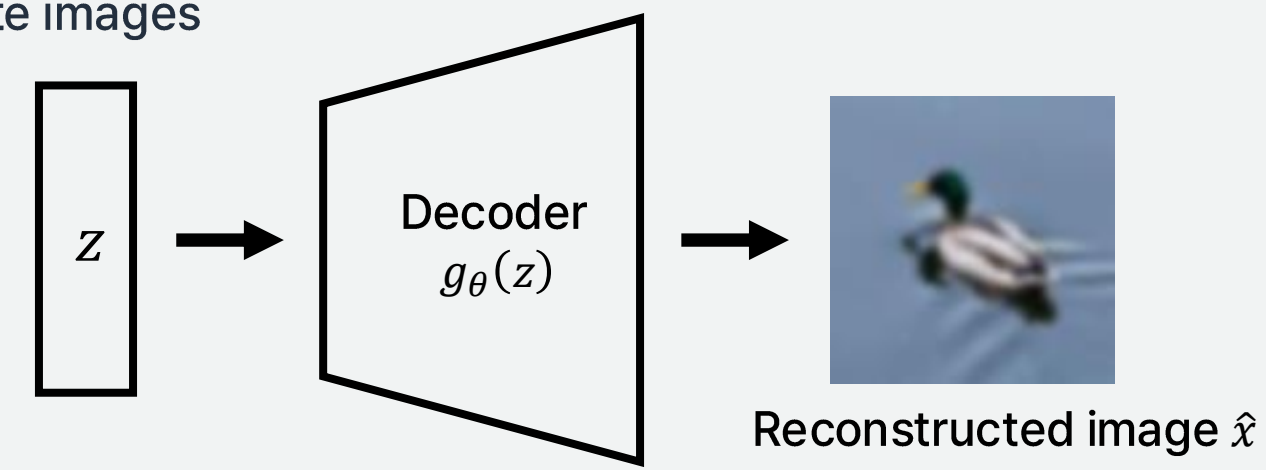


However, we do not know
the probability distribution of z

Can we make the P(z) nicer?
- e.g., p(z)=N(0,I)

Autoencoder

- How can we generate new data from this?
  - Generate a new z to use decoder to generate images

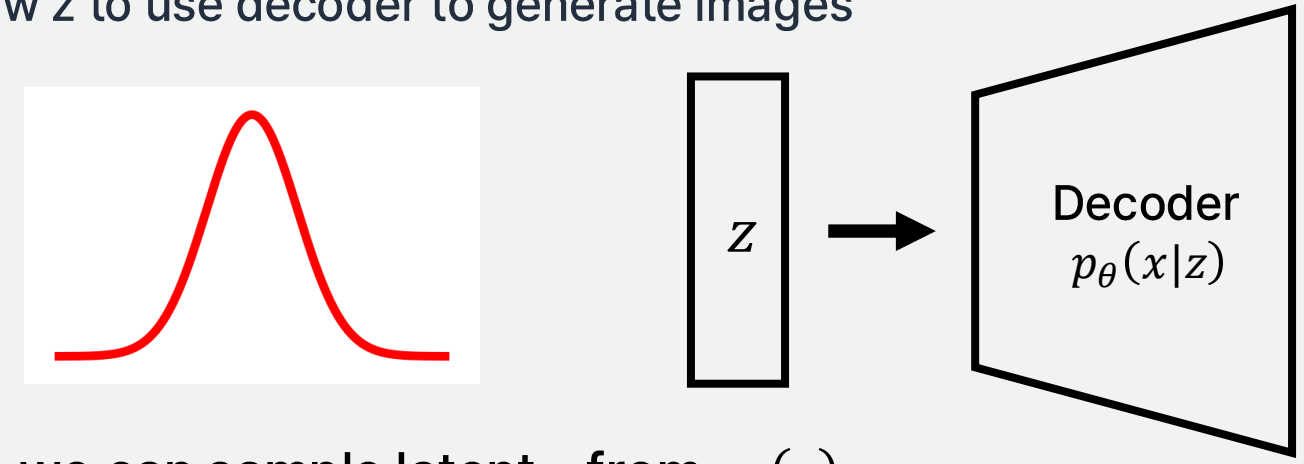


However, we do not know
the probability distribution of z

Can we make the P(z) nicer?
- e.g., p(z)=N(0,I)
- It is easy to sample from a Gaussian

Autoencoder

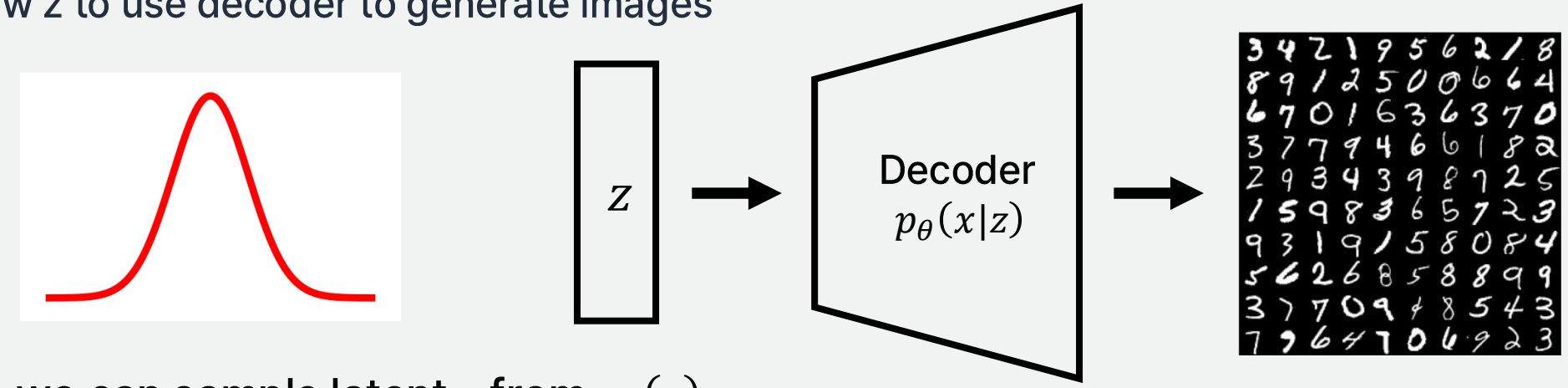
- How can we generate new data from this?
  - Generate a new z to use decoder to generate images



If $p_\theta(z) = \mathcal{N}(0, \mathrm{I})$, we can sample latent z from $p_\theta(z)$

Autoencoder

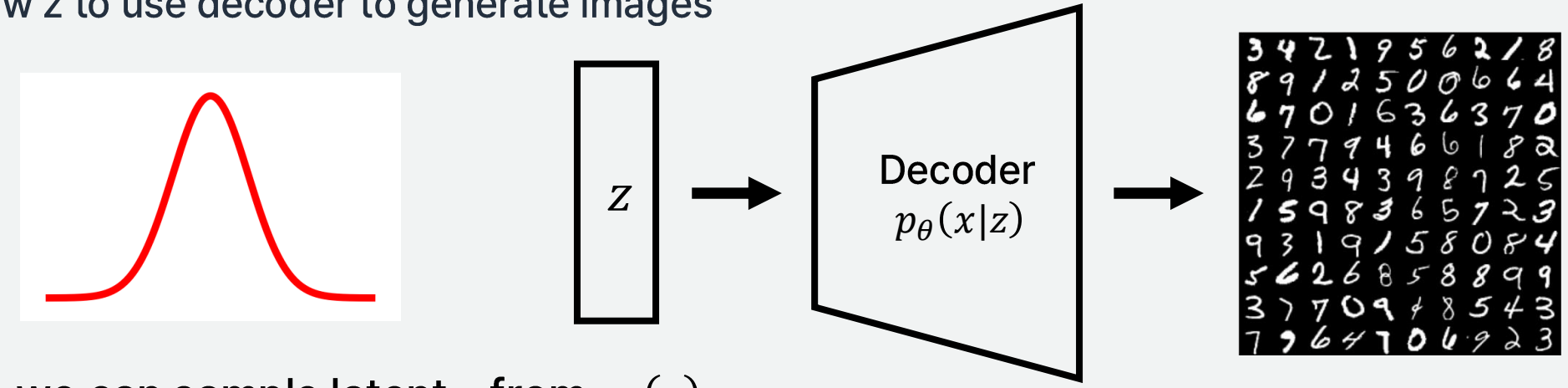
- How can we generate new data from this?
  - Generate a new z to use decoder to generate images



If $p_\theta(z) = \mathcal{N}(0, \mathrm{I})$, we can sample latent z from $p_\theta(z)$
Then, we can sample x from $p_\theta(x|z)$

Autoencoder

- How can we generate new data from this?
  - Generate a new z to use decoder to generate images

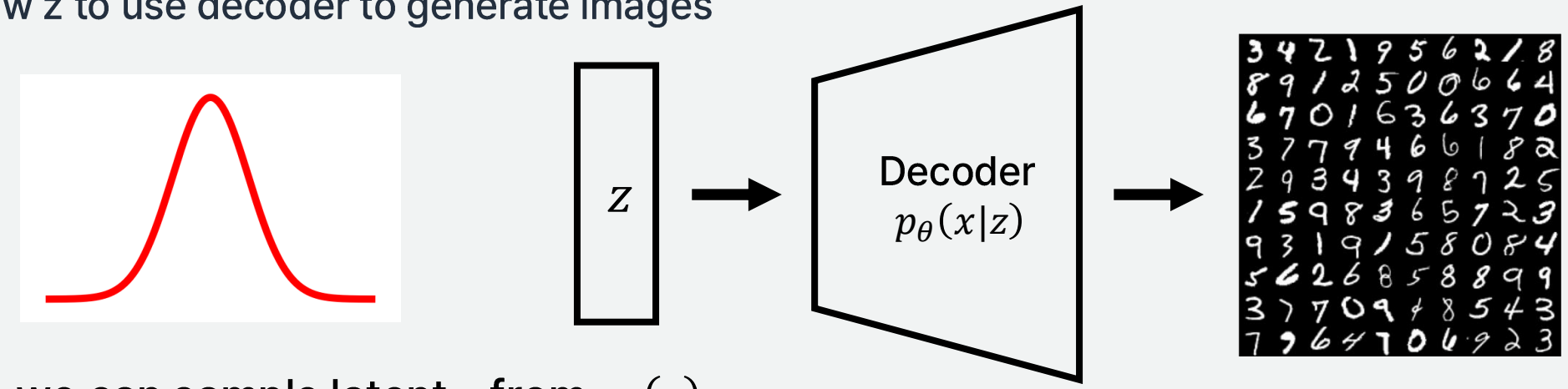


If $p_\theta(z) = \mathcal{N}(0, \mathrm{I})$, we can sample latent z from $p_\theta(z)$
Then, we can sample x from $p_\theta(x|z)$

How to train $p_\theta(x|z)$?

Autoencoder

- How can we generate new data from this?
  - Generate a new z to use decoder to generate images



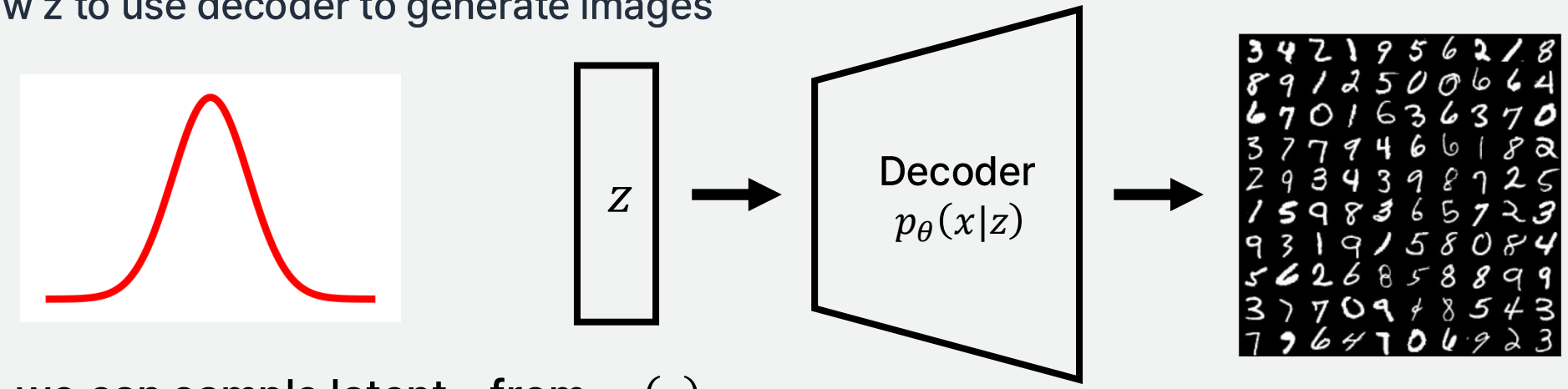
If $p_\theta(z) = \mathcal{N}(0, \mathrm{I})$, we can sample latent z from $p_\theta(z)$
Then, we can sample x from $p_\theta(x|z)$

How to train $p_\theta(x|z)$?

$$p_\theta(x)$$

Autoencoder

- How can we generate new data from this?
  - Generate a new z to use decoder to generate images



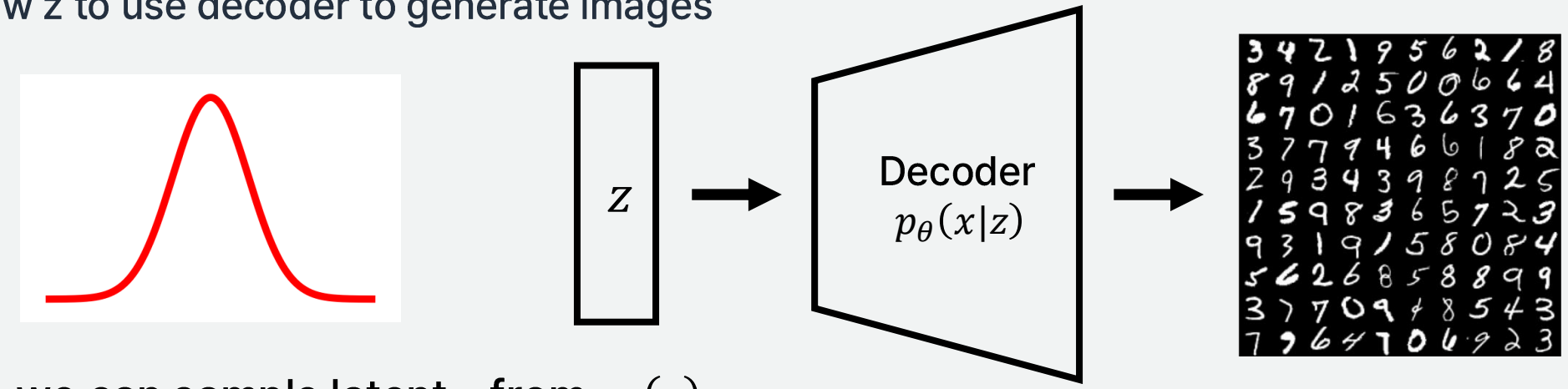
If $p_\theta(z) = \mathcal{N}(0, \mathrm{I})$, we can sample latent z from $p_\theta(z)$
Then, we can sample x from $p_\theta(x|z)$

How to train $p_\theta(x|z)$?

$$p_\theta(x) = \int p_\theta(z) p_\theta(x|z) dz$$

Autoencoder

- How can we generate new data from this?
  - Generate a new z to use decoder to generate images



If $p_\theta(z) = \mathcal{N}(0, \mathrm{I})$, we can sample latent z from $p_\theta(z)$
Then, we can sample x from $p_\theta(x|z)$

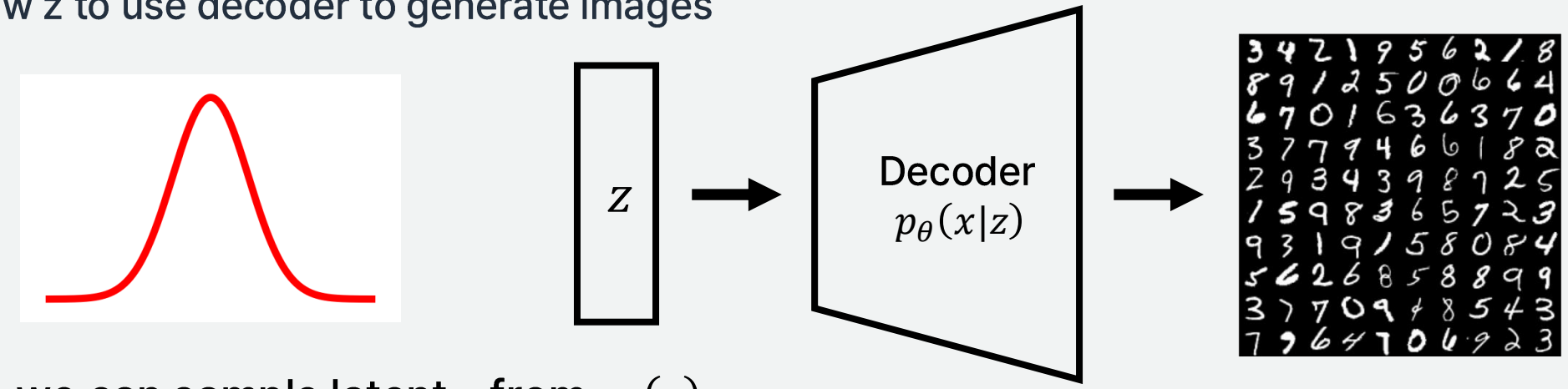
How to train $p_\theta(x|z)$?

$$p_\theta(x) = \int p_\theta(z) p_\theta(x|z) dz$$

Gaussian; easy to compute

Autoencoder

- How can we generate new data from this?
  - Generate a new z to use decoder to generate images



If $p_\theta(z) = \mathcal{N}(0, \mathrm{I})$, we can sample latent z from $p_\theta(z)$
Then, we can sample x from $p_\theta(x|z)$

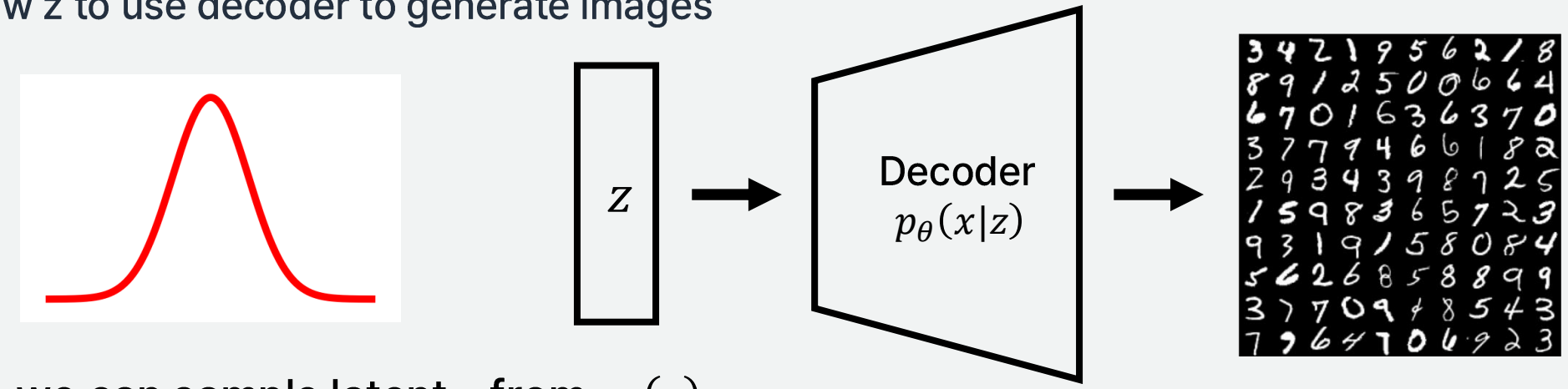
How to train $p_\theta(x|z)$?

$$p_\theta(x) = \int p_\theta(z) p_\theta(x|z) dz$$

We can compute it using decoder

Autoencoder

- How can we generate new data from this?
  - Generate a new z to use decoder to generate images



If $p_\theta(z) = \mathcal{N}(0, \mathrm{I})$, we can sample latent z from $p_\theta(z)$
Then, we can sample x from $p_\theta(x|z)$

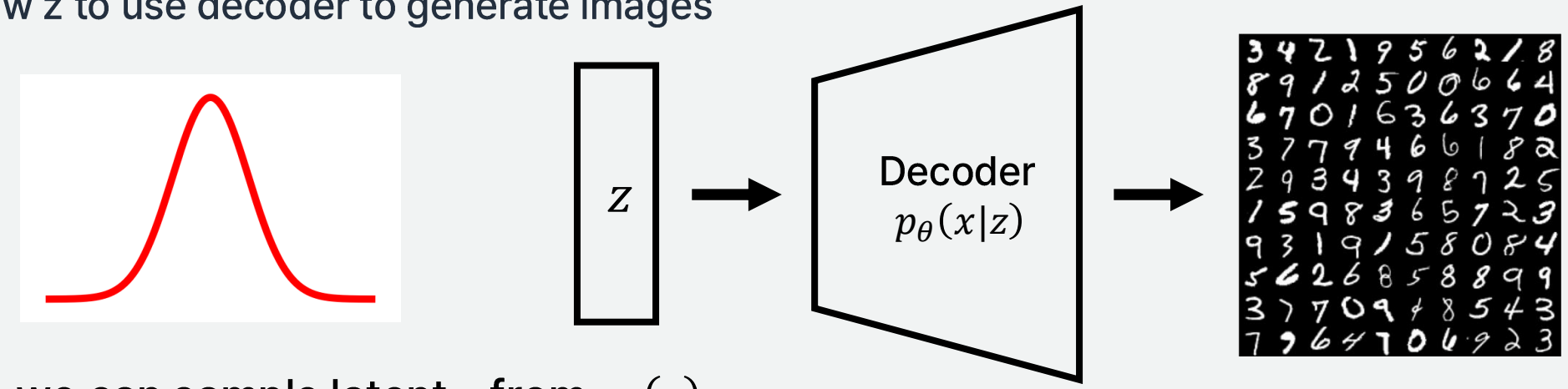
How to train $p_\theta(x|z)$?

$$p_\theta(x) = \int p_\theta(z) p_\theta(x|z) dz$$

Intractable to integrate over z

Autoencoder

- How can we generate new data from this?
  - Generate a new z to use decoder to generate images



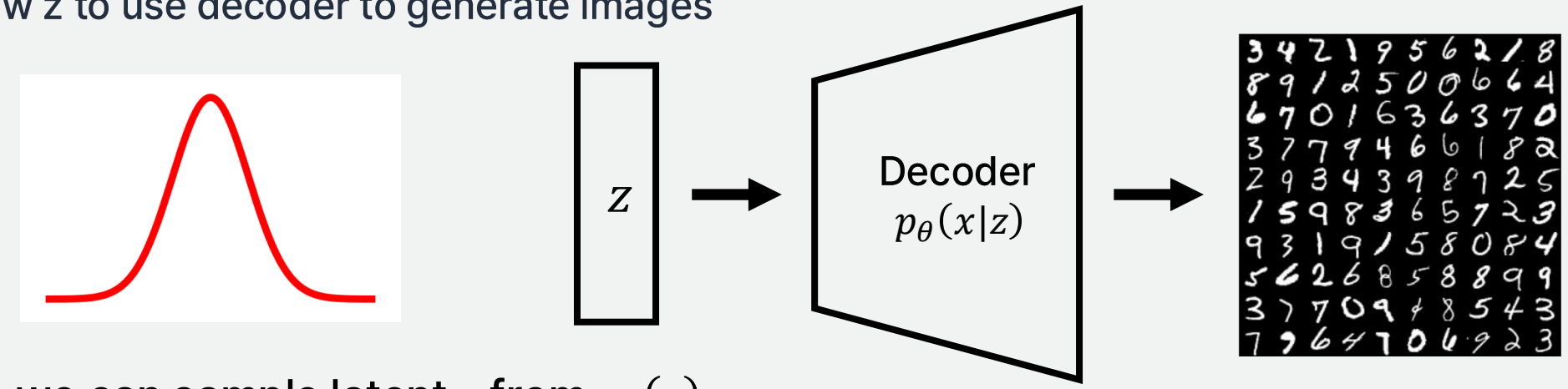
If $p_\theta(z) = \mathcal{N}(0, \mathrm{I})$, we can sample latent z from $p_\theta(z)$
Then, we can sample x from $p_\theta(x|z)$

How to train $p_\theta(x|z)$?

$$p_\theta(x) = \frac{p_\theta(x|z)p_\theta(z)}{p_\theta(z|x)}$$

Autoencoder

- How can we generate new data from this?
  - Generate a new z to use decoder to generate images



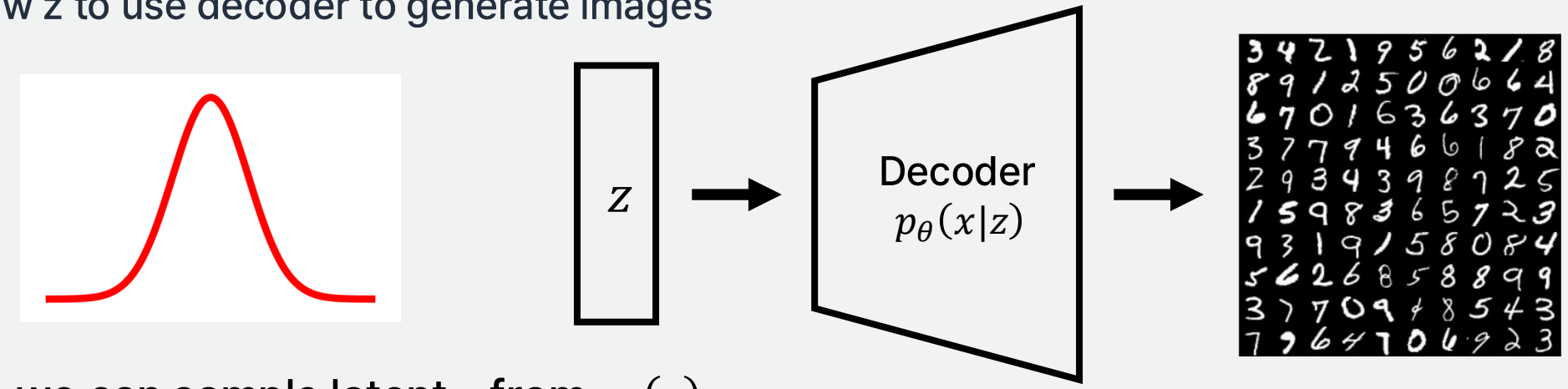
If $p_\theta(z) = \mathcal{N}(0, \mathrm{I})$, we can sample latent z from $p_\theta(z)$
Then, we can sample x from $p_\theta(x|z)$

How to train $p_\theta(x|z)$?

$$p_\theta(x) = \frac{p_\theta(x|z)p_\theta(z)}{p_\theta(z|x)}$$ → easy to compute

Autoencoder

- How can we generate new data from this?
  - Generate a new z to use decoder to generate images



If $p_\theta(z) = \mathcal{N}(0, \mathrm{I})$, we can sample latent z from $p_\theta(z)$
Then, we can sample x from $p_\theta(x|z)$

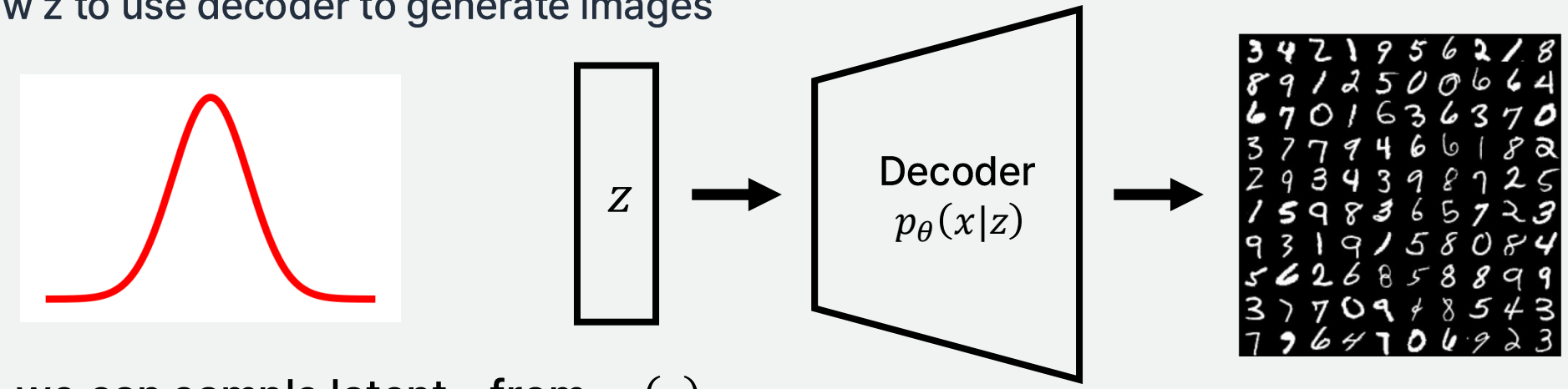
How to train $p_\theta(x|z)$?

$$p_\theta(x) = \frac{p_\theta(x|z)p_\theta(z)}{p_\theta(z|x)}$$

However, we do not know this

Autoencoder

- How can we generate new data from this?
  - Generate a new z to use decoder to generate images



If $p_\theta(z) = \mathcal{N}(0, \mathrm{I})$, we can sample latent z from $p_\theta(z)$
Then, we can sample x from $p_\theta(x|z)$

How to train $p_\theta(x|z)$?

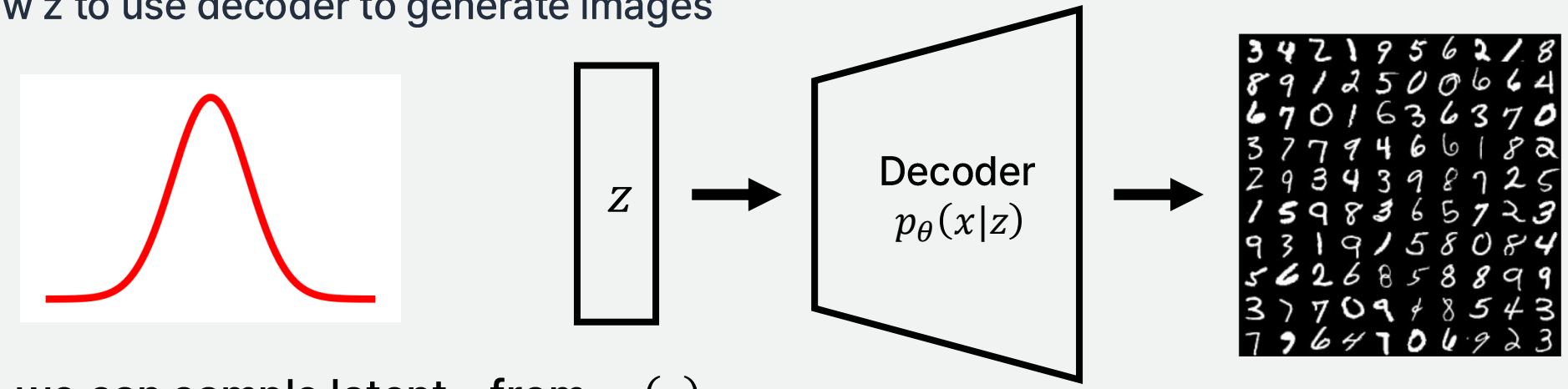
$$p_\theta(x) = \frac{p_\theta(x|z)p_\theta(z)}{p_\theta(z|x)}$$

However, we do not know this

Approximate it using $q_\phi(z|x)$

Autoencoder

- How can we generate new data from this?
  - Generate a new z to use decoder to generate images



If $p_\theta(z) = \mathcal{N}(0, \mathrm{I})$, we can sample latent z from $p_\theta(z)$
Then, we can sample x from $p_\theta(x|z)$

How to train $p_\theta(x|z)$?

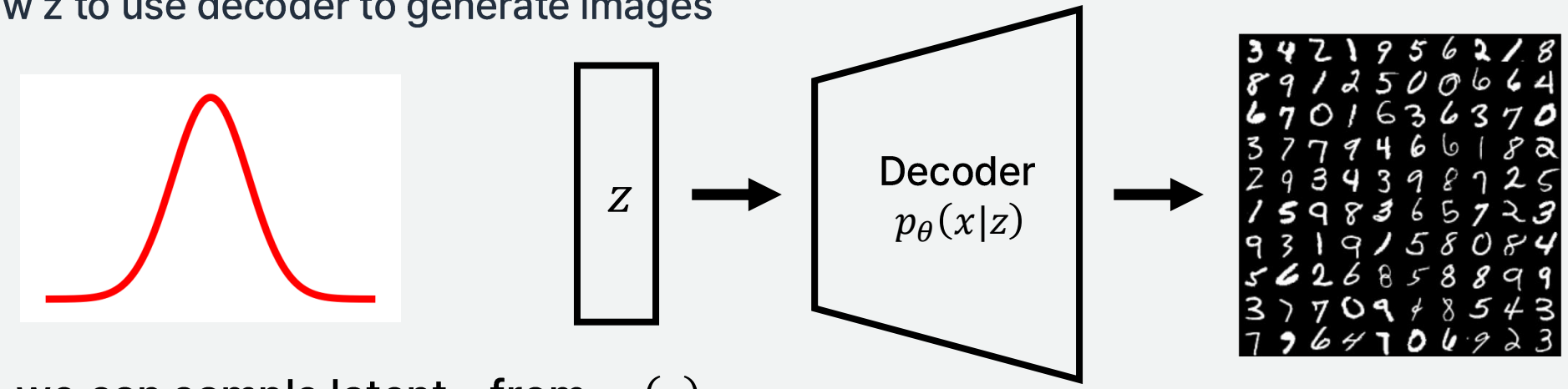
$$p_\theta(x) = \frac{p_\theta(x|z)p_\theta(z)}{p_\theta(z|x)}$$

Approximate it using $q_\phi(z|x)$

How do we approximate it?

Autoencoder

- How can we generate new data from this?
  - Generate a new z to use decoder to generate images



If $p_\theta(z) = \mathcal{N}(0, \mathrm{I})$, we can sample latent z from $p_\theta(z)$
Then, we can sample x from $p_\theta(x|z)$

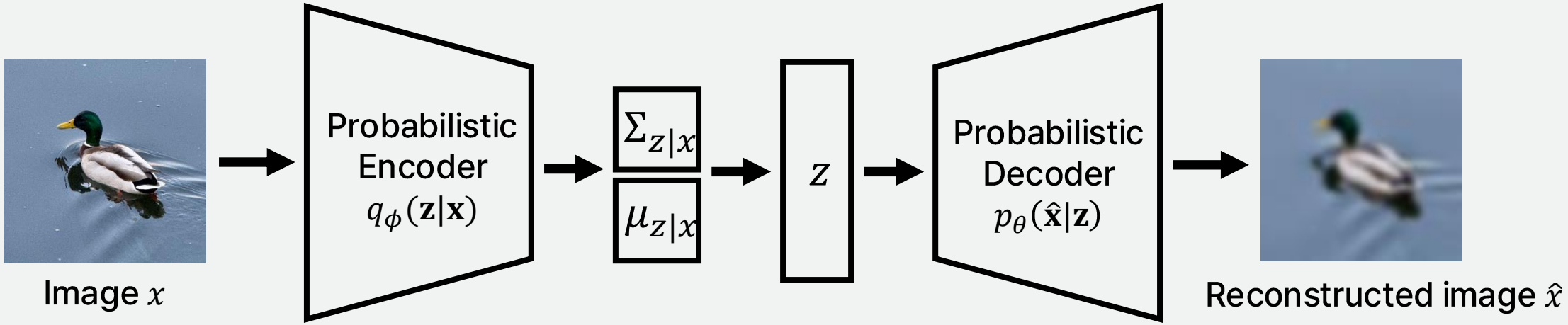
How to train $p_\theta(x|z)$?

$$p_\theta(x) = \frac{p_\theta(x|z)p_\theta(z)}{p_\theta(z|x)}$$

Approximate it using $q_\phi(z|x)$

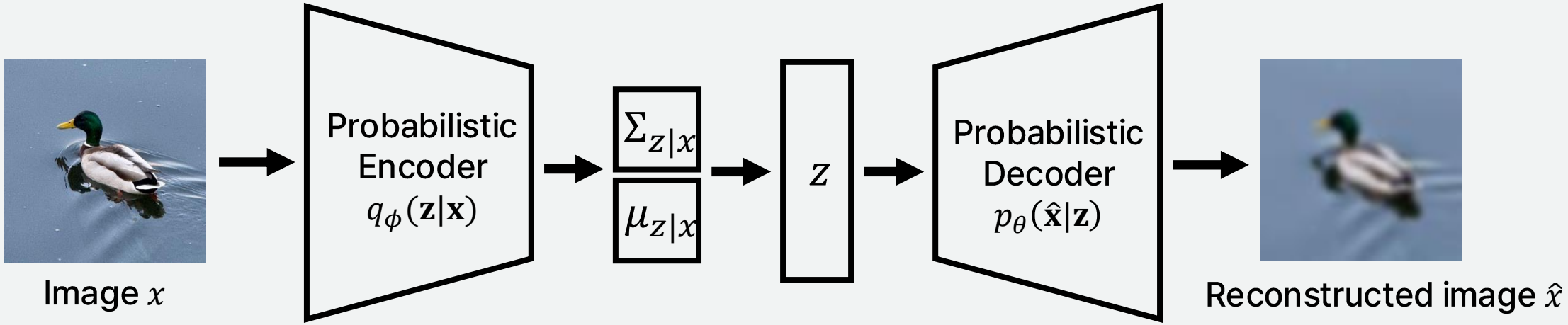
How do we approximate it?
Use Encoder as $q_\phi(z|x)$

Variational Autoencoder



Kingma et al., "Auto-encoding Variational Bayes", ICLR 2014.

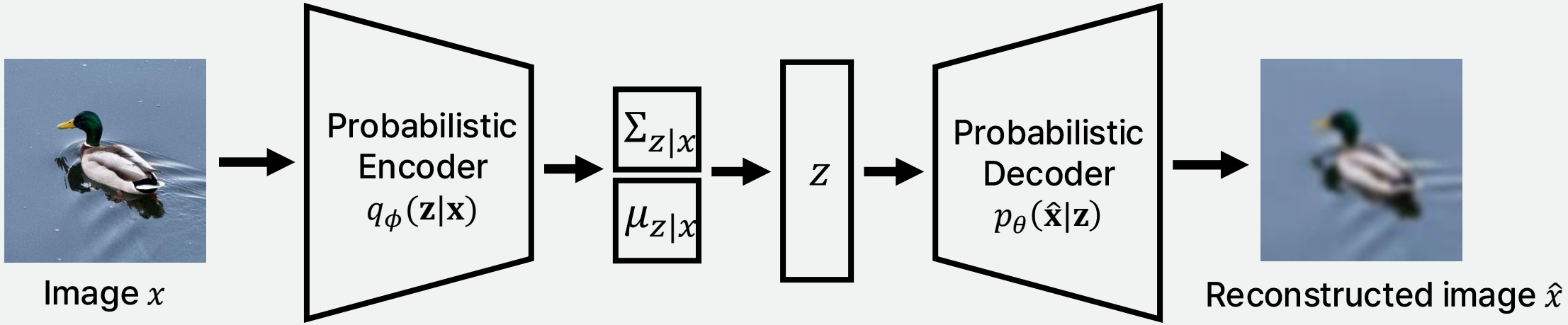
Variational Autoencoder



How do we train this?

Kingma et al., "Auto-encoding Variational Bayes", ICLR 2014.

Variational Autoencoder



How do we train this? Maximize $p(x)$

Kingma et al., "Auto-encoding Variational Bayes", ICLR 2014.

Variational Autoencoder

$$\log p_\theta(x)$$

Kingma et al., “Auto-encoding Variational Bayes”, ICLR 2014.

Variational Autoencoder

$$\log p_\theta(x) = \log \frac{p_\theta(x|z)p(z)}{p_\theta(z|x)}$$

By Bayes' Rule

Kingma et al., "Auto-encoding Variational Bayes", ICLR 2014.

Variational Autoencoder

$$\log p_\theta(x) = \log \frac{p_\theta(x|z)p(z)}{p_\theta(z|x)} = \log \frac{p_\theta(x|z)p(z)q_\phi(z|x)}{p_\theta(z|x)q_\phi(z|x)}$$

Kingma et al., “Auto-encoding Variational Bayes”, ICLR 2014.

Variational Autoencoder

$$\log p_\theta(x) = \log \frac{p_\theta(x|z)p(z)}{p_\theta(z|x)} = \log \frac{p_\theta(x|z)p(z)q_\phi(z|x)}{p_\theta(z|x)q_\phi(z|x)}$$

$$= \log p_\theta(x|z) - \log \frac{q_\phi(z|x)}{p(z)} + \log \frac{q_\phi(z|x)}{p_\theta(z|x)}$$

Kingma et al., “Auto-encoding Variational Bayes”, ICLR 2014.

Variational Autoencoder

$$\log p_\theta(x) = \mathbb{E}_{z\sim q_\phi}\left[\log \frac{p_\theta(x|z)p(z)}{p_\theta(z|x)}\right] = \mathbb{E}_{z\sim q_\phi}\left[\log \frac{p_\theta(x|z)p(z)q_\phi(z|x)}{p_\theta(z|x)q_\phi(z|x)}\right]$$

$$= \mathbb{E}_{z\sim q_\phi}\left[\log p_\theta(x|z) - \log \frac{q_\phi(z|x)}{p(z)} + \log \frac{q_\phi(z|x)}{p_\theta(z|x)}\right]$$

Average over $z\sim q_\phi$, since $p_\theta(x)$ does not depend on z

Kingma et al., “Auto-encoding Variational Bayes”, ICLR 2014.

Variational Autoencoder

$$\log p_\theta(x) = \mathbb{E}_{z\sim q_\phi}\left[\log \frac{p_\theta(x|z)p(z)}{p_\theta(z|x)}\right] = \mathbb{E}_{z\sim q_\phi}\left[\log \frac{p_\theta(x|z)p(z)q_\phi(z|x)}{p_\theta(z|x)q_\phi(z|x)}\right]$$

$$= \mathbb{E}_{z\sim q_\phi}\left[\log p_\theta(x|z) - \log \frac{q_\phi(z|x)}{p(z)} + \log \frac{q_\phi(z|x)}{p_\theta(z|x)}\right]$$

$$= \mathbb{E}_{z\sim q_\phi}[\log p_\theta(x|z)] - \mathbb{E}_{z\sim q_\phi}\left[\log \frac{q_\phi(z|x)}{p(z)}\right] + \mathbb{E}_{z\sim q_\phi}\left[\log \frac{q_\phi(z|x)}{p_\theta(z|x)}\right]$$

Kingma et al., “Auto-encoding Variational Bayes”, ICLR 2014.

Variational Autoencoder

$$\log p_\theta(x) = \mathbb{E}_{z\sim q_\phi}\left[\log \frac{p_\theta(x|z)p(z)}{p_\theta(z|x)}\right] = \mathbb{E}_{z\sim q_\phi}\left[\log \frac{p_\theta(x|z)p(z)q_\phi(z|x)}{p_\theta(z|x)q_\phi(z|x)}\right]$$

$$= \mathbb{E}_{z\sim q_\phi}\left[\log p_\theta(x|z) - \log \frac{q_\phi(z|x)}{p(z)} + \log \frac{q_\phi(z|x)}{p_\theta(z|x)}\right]$$

$$= \mathbb{E}_{z\sim q_\phi}[\log p_\theta(x|z)] - \mathbb{E}_{z\sim q_\phi}\left[\log \frac{q_\phi(z|x)}{p(z)}\right] + \mathbb{E}_{z\sim q_\phi}\left[\log \frac{q_\phi(z|x)}{p_\theta(z|x)}\right]$$

$$= \mathbb{E}_{z\sim q_\phi}[\log p_\theta(x|z)] - D_{KL}\left(q_\phi(z|x)||p(z)\right) + D_{KL}\left(q_\phi(z|x)||p_\theta(z|x)\right)$$

Kingma et al., “Auto-encoding Variational Bayes”, ICLR 2014.

Variational Autoencoder

$$\log p_\theta(x) = \mathbb{E}_{z\sim q_\phi}\left[\log \frac{p_\theta(x|z)p(z)}{p_\theta(z|x)}\right] = \mathbb{E}_{z\sim q_\phi}\left[\log \frac{p_\theta(x|z)p(z)q_\phi(z|x)}{p_\theta(z|x)q_\phi(z|x)}\right]$$

$$= \mathbb{E}_{z\sim q_\phi}\left[\log p_\theta(x|z) - \log \frac{q_\phi(z|x)}{p(z)} + \log \frac{q_\phi(z|x)}{p_\theta(z|x)}\right]$$

$$= \mathbb{E}_{z\sim q_\phi}[\log p_\theta(x|z)] - \mathbb{E}_{z\sim q_\phi}\left[\log \frac{q_\phi(z|x)}{p(z)}\right] + \mathbb{E}_{z\sim q_\phi}\left[\log \frac{q_\phi(z|x)}{p_\theta(z|x)}\right]$$

$$= \underline{\mathbb{E}_{z\sim q_\phi}[\log p_\theta(x|z)]} - D_{KL}\left(q_\phi(z|x)||p(z)\right) + D_{KL}\left(q_\phi(z|x)||p_\theta(z|x)\right)$$

Data Reconstruction Error

Kingma et al., “Auto-encoding Variational Bayes”, ICLR 2014.

Variational Autoencoder

$$\log p_\theta(x) = \mathbb{E}_{z\sim q_\phi}\left[\log \frac{p_\theta(x|z)p(z)}{p_\theta(z|x)}\right] = \mathbb{E}_{z\sim q_\phi}\left[\log \frac{p_\theta(x|z)p(z)q_\phi(z|x)}{p_\theta(z|x)q_\phi(z|x)}\right]$$

$$= \mathbb{E}_{z\sim q_\phi}\left[\log p_\theta(x|z) - \log \frac{q_\phi(z|x)}{p(z)} + \log \frac{q_\phi(z|x)}{p_\theta(z|x)}\right]$$

$$= \mathbb{E}_{z\sim q_\phi}[\log p_\theta(x|z)] - \mathbb{E}_{z\sim q_\phi}\left[\log \frac{q_\phi(z|x)}{p(z)}\right] + \mathbb{E}_{z\sim q_\phi}\left[\log \frac{q_\phi(z|x)}{p_\theta(z|x)}\right]$$

$$= \mathbb{E}_{z\sim q_\phi}[\log p_\theta(x|z)] - \underline{D_{KL}\left(q_\phi(z|x)||p(z)\right)} + D_{KL}\left(q_\phi(z|x)||p_\theta(z|x)\right)$$

Distribution KL divergence

Kingma et al., "Auto-encoding Variational Bayes", ICLR 2014.

Variational Autoencoder

$$\log p_\theta(x) = \mathbb{E}_{z\sim q_\phi}\left[\log \frac{p_\theta(x|z)p(z)}{p_\theta(z|x)}\right] = \mathbb{E}_{z\sim q_\phi}\left[\log \frac{p_\theta(x|z)p(z)q_\phi(z|x)}{p_\theta(z|x)q_\phi(z|x)}\right]$$

$$= \mathbb{E}_{z\sim q_\phi}\left[\log p_\theta(x|z) - \log \frac{q_\phi(z|x)}{p(z)} + \log \frac{q_\phi(z|x)}{p_\theta(z|x)}\right]$$

$$= \mathbb{E}_{z\sim q_\phi}[\log p_\theta(x|z)] - \mathbb{E}_{z\sim q_\phi}\left[\log \frac{q_\phi(z|x)}{p(z)}\right] + \mathbb{E}_{z\sim q_\phi}\left[\log \frac{q_\phi(z|x)}{p_\theta(z|x)}\right]$$

$$= \mathbb{E}_{z\sim q_\phi}[\log p_\theta(x|z)] - D_{KL}\left(q_\phi(z|x)||p(z)\right) + \underbrace{D_{KL}\left(q_\phi(z|x)||p_\theta(z|x)\right)}_{p_\theta(z|x) \text{ is intractable}}$$

Kingma et al., "Auto-encoding Variational Bayes", ICLR 2014.

Variational Autoencoder

$$\log p_\theta(x) = \mathbb{E}_{z\sim q_\phi}\left[\log \frac{p_\theta(x|z)p(z)}{p_\theta(z|x)}\right] = \mathbb{E}_{z\sim q_\phi}\left[\log \frac{p_\theta(x|z)p(z)q_\phi(z|x)}{p_\theta(z|x)q_\phi(z|x)}\right]$$

$$= \mathbb{E}_{z\sim q_\phi}\left[\log p_\theta(x|z) - \log \frac{q_\phi(z|x)}{p(z)} + \log \frac{q_\phi(z|x)}{p_\theta(z|x)}\right]$$

$$= \mathbb{E}_{z\sim q_\phi}[\log p_\theta(x|z)] - \mathbb{E}_{z\sim q_\phi}\left[\log \frac{q_\phi(z|x)}{p(z)}\right] + \mathbb{E}_{z\sim q_\phi}\left[\log \frac{q_\phi(z|x)}{p_\theta(z|x)}\right]$$

$$= \mathbb{E}_{z\sim q_\phi}[\log p_\theta(x|z)] - D_{KL}\left(q_\phi(z|x)||p(z)\right) + \underline{D_{KL}\left(q_\phi(z|x)||p_\theta(z|x)\right)}$$

$p_\theta(z|x)$ is intractable

$$p_\theta(z|x) = \frac{p_\theta(x|z)p_\theta(z)}{p_\theta(x)}$$

Kingma et al., "Auto-encoding Variational Bayes", ICLR 2014.

Variational Autoencoder

$$\log p_\theta(x) = \mathbb{E}_{z\sim q_\phi}\left[\log \frac{p_\theta(x|z)p(z)}{p_\theta(z|x)}\right] = \mathbb{E}_{z\sim q_\phi}\left[\log \frac{p_\theta(x|z)p(z)q_\phi(z|x)}{p_\theta(z|x)q_\phi(z|x)}\right]$$

$$= \mathbb{E}_{z\sim q_\phi}\left[\log p_\theta(x|z) - \log \frac{q_\phi(z|x)}{p(z)} + \log \frac{q_\phi(z|x)}{p_\theta(z|x)}\right]$$

$$= \mathbb{E}_{z\sim q_\phi}[\log p_\theta(x|z)] - \mathbb{E}_{z\sim q_\phi}\left[\log \frac{q_\phi(z|x)}{p(z)}\right] + \mathbb{E}_{z\sim q_\phi}\left[\log \frac{q_\phi(z|x)}{p_\theta(z|x)}\right]$$

$$= \mathbb{E}_{z\sim q_\phi}[\log p_\theta(x|z)] - D_{KL}\left(q_\phi(z|x)||p(z)\right) + D_{KL}\left(q_\phi(z|x)||p_\theta(z|x)\right)$$

$p_\theta(z|x)$ is intractable
but since $D_{KL} \geq 0$,
$\log p_\theta(x)$ is lower-bounded

Kingma et al., "Auto-encoding Variational Bayes", ICLR 2014.

Variational Autoencoder

$$\log p_\theta(x) = \mathbb{E}_{z\sim q_\phi}\left[\log \frac{p_\theta(x|z)p(z)}{p_\theta(z|x)}\right] = \mathbb{E}_{z\sim q_\phi}\left[\log \frac{p_\theta(x|z)p(z)q_\phi(z|x)}{p_\theta(z|x)q_\phi(z|x)}\right]$$

$$= \mathbb{E}_{z\sim q_\phi}\left[\log p_\theta(x|z) - \log \frac{q_\phi(z|x)}{p(z)} + \log \frac{q_\phi(z|x)}{p_\theta(z|x)}\right]$$

$$= \mathbb{E}_{z\sim q_\phi}[\log p_\theta(x|z)] - \mathbb{E}_{z\sim q_\phi}\left[\log \frac{q_\phi(z|x)}{p(z)}\right] + \mathbb{E}_{z\sim q_\phi}\left[\log \frac{q_\phi(z|x)}{p_\theta(z|x)}\right]$$

$$= \mathbb{E}_{z\sim q_\phi}[\log p_\theta(x|z)] - D_{KL}\left(q_\phi(z|x)||p(z)\right) + D_{KL}\left(q_\phi(z|x)||p_\theta(z|x)\right)$$

$$\geq \mathbb{E}_{z\sim q_\phi}[\log p_\theta(x|z)] - D_{KL}\left(q_\phi(z|x)||p(z)\right)$$

Kingma et al., “Auto-encoding Variational Bayes”, ICLR 2014.

Variational Autoencoder

$$\log p_\theta(x) = \mathbb{E}_{z\sim q_\phi}\left[\log \frac{p_\theta(x|z)p(z)}{p_\theta(z|x)}\right] = \mathbb{E}_{z\sim q_\phi}\left[\log \frac{p_\theta(x|z)p(z)q_\phi(z|x)}{p_\theta(z|x)q_\phi(z|x)}\right]$$

$$= \mathbb{E}_{z\sim q_\phi}\left[\log p_\theta(x|z) - \log \frac{q_\phi(z|x)}{p(z)} + \log \frac{q_\phi(z|x)}{p_\theta(z|x)}\right]$$

$$= \mathbb{E}_{z\sim q_\phi}[\log p_\theta(x|z)] - \mathbb{E}_{z\sim q_\phi}\left[\log \frac{q_\phi(z|x)}{p(z)}\right] + \mathbb{E}_{z\sim q_\phi}\left[\log \frac{q_\phi(z|x)}{p_\theta(z|x)}\right]$$

$$= \mathbb{E}_{z\sim q_\phi}[\log p_\theta(x|z)] - D_{KL}\left(q_\phi(z|x)||p(z)\right) + D_{KL}\left(q_\phi(z|x)||p_\theta(z|x)\right)$$

$$\geq \mathbb{E}_{z\sim q_\phi}[\log p_\theta(x|z)] - D_{KL}\left(q_\phi(z|x)||p(z)\right)$$ ELBO (Evidence Lower Bound)

Kingma et al., "Auto-encoding Variational Bayes", ICLR 2014.

Variational Autoencoder

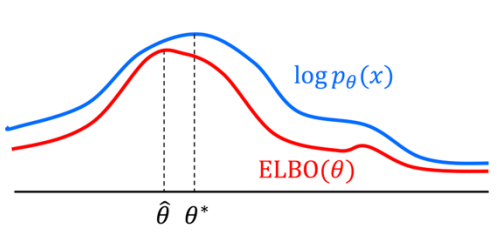
$$\log p_\theta(x) = \mathbb{E}_{z\sim q_\phi}\left[\log \frac{p_\theta(x|z)p(z)}{p_\theta(z|x)}\right] = \mathbb{E}_{z\sim q_\phi}\left[\log \frac{p_\theta(x|z)p(z)q_\phi(z|x)}{p_\theta(z|x)q_\phi(z|x)}\right]$$

$$= \mathbb{E}_{z\sim q_\phi}\left[\log p_\theta(x|z) - \log \frac{q_\phi(z|x)}{p(z)} + \log \frac{q_\phi(z|x)}{p_\theta(z|x)}\right]$$

$$= \mathbb{E}_{z\sim q_\phi}[\log p_\theta(x|z)] - \mathbb{E}_{z\sim q_\phi}\left[\log \frac{q_\phi(z|x)}{p(z)}\right] + \mathbb{E}_{z\sim q_\phi}\left[\log \frac{q_\phi(z|x)}{p_\theta(z|x)}\right]$$

$$= \mathbb{E}_{z\sim q_\phi}[\log p_\theta(x|z)] - D_{KL}\left(q_\phi(z|x)||p(z)\right) + D_{KL}\left(q_\phi(z|x)||p_\theta(z|x)\right)$$

$$\geq \mathbb{E}_{z\sim q_\phi}[\log p_\theta(x|z)] - D_{KL}\left(q_\phi(z|x)||p(z)\right)$$



Kingma et al., "Auto-encoding Variational Bayes", ICLR 2014.

Image from: https://www.math.emory.edu/site/cmds-reuret/projects/2024-vae/

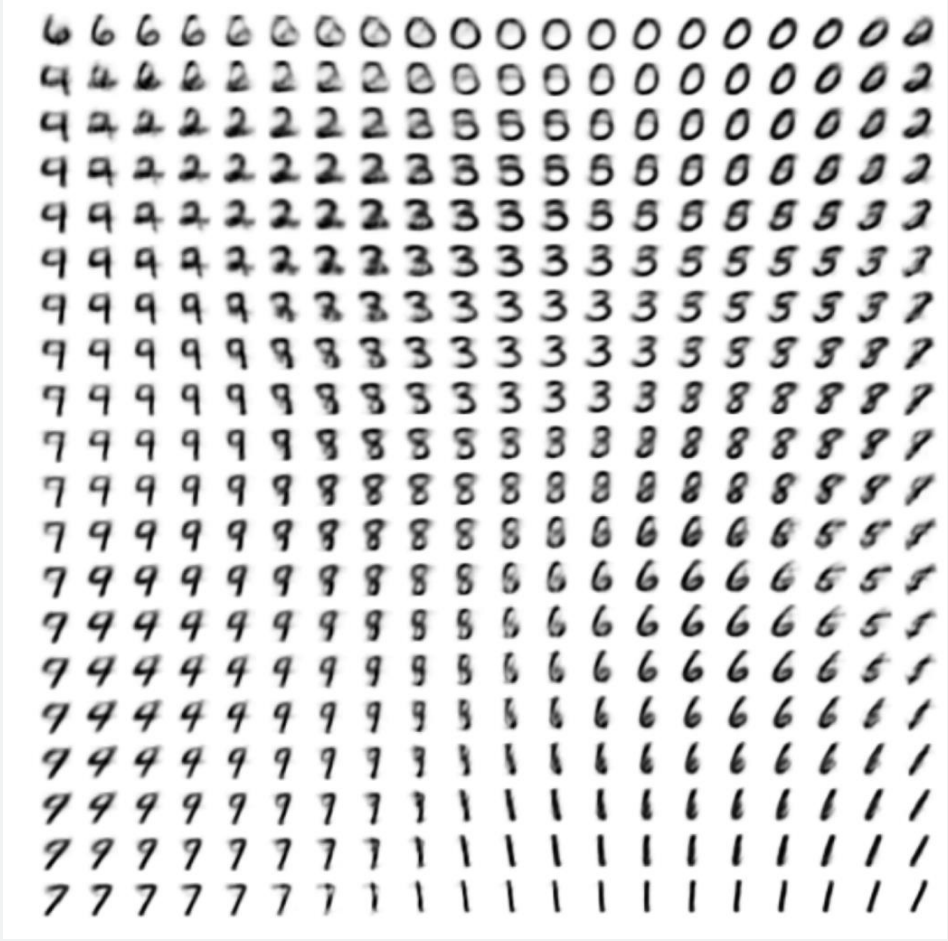
VAE Generated Samples



Kingma et al., “Auto-encoding Variational Bayes”, ICLR 2014.

Latent space interpolation



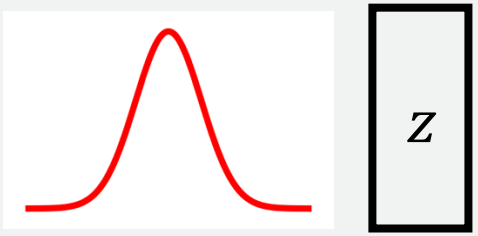


Kingma et al., “Auto-encoding Variational Bayes”, ICLR 2014.

Generative Adversarial Networks (GANs)

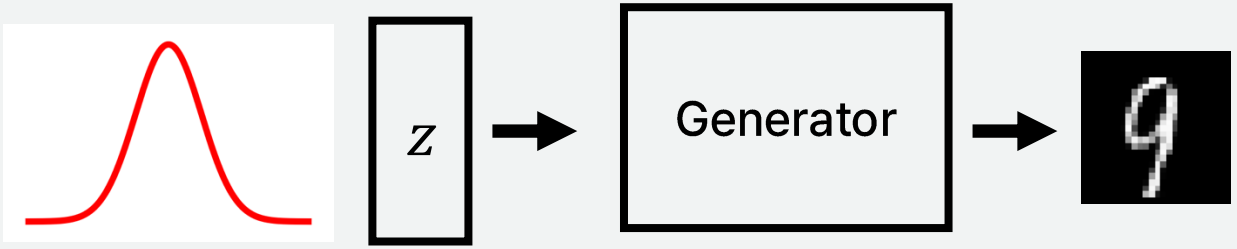
Generate without Explicit $p(x)$

- Idea: Generate x directly from a simple prior $p(z)$



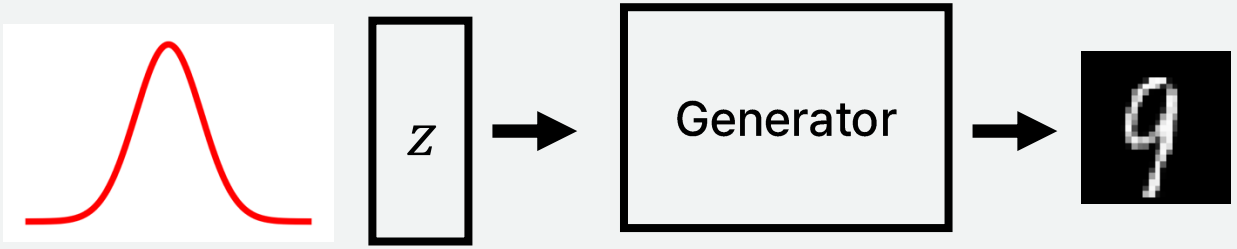
Generate without Explicit $p(x)$

- Idea: Generate x directly from a simple prior $p(z)$
  - e.g., use a neural network $G(z)$ to generate $\hat{x}$



Generate without Explicit $p(x)$

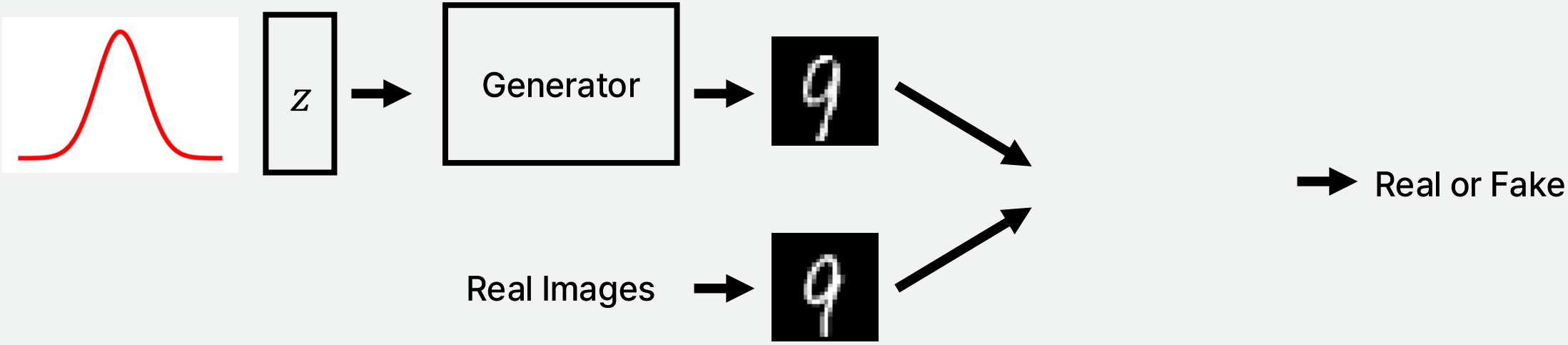
- Idea: Generate x directly from a simple prior $p(z)$
  - e.g., use a neural network $G(z)$ to generate $\hat{x}$
  - Problem: how do we know $\hat{x}$ is realistic without $p(x)$?



Generate without Explicit $p(x)$

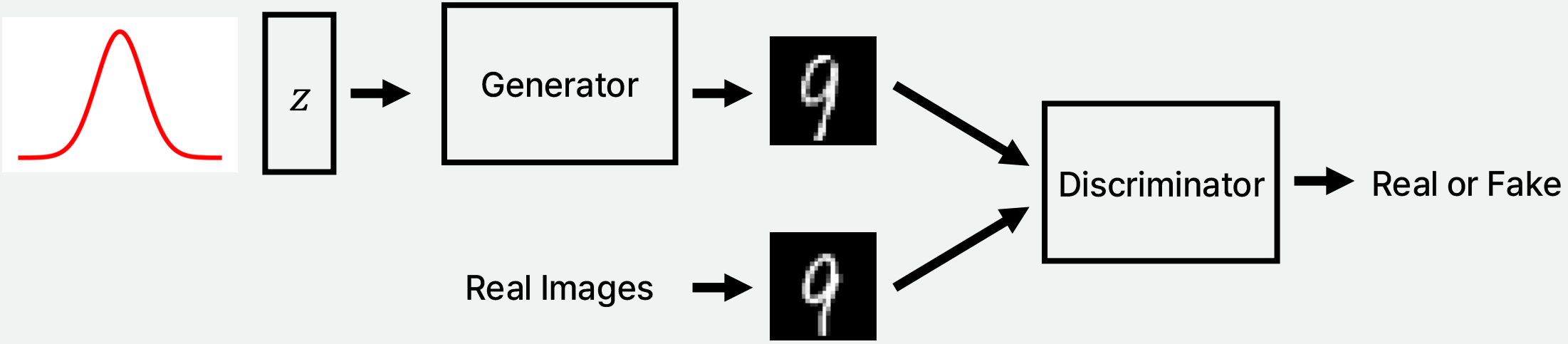
- Idea: Generate x directly from a simple prior $p(z)$
  - e.g., use a neural network $G(z)$ to generate $\hat{x}$
  - Problem: how do we know $\hat{x}$ is realistic without $p(x)$?

distinguish it from real data

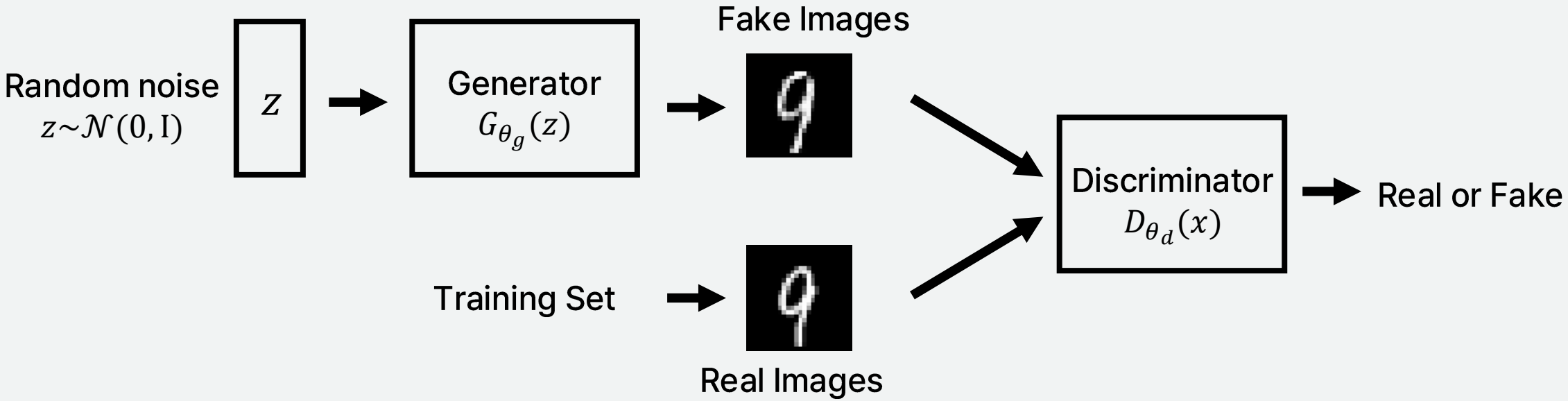


Generate without Explicit $p(x)$

- Idea: Generate x directly from a simple prior $p(z)$
  - e.g., use a neural network $G(z)$ to generate $\hat{x}$
  - Problem: how do we know $\hat{x}$ is realistic without $p(x)$?
  - Use another network to distinguish it from real data

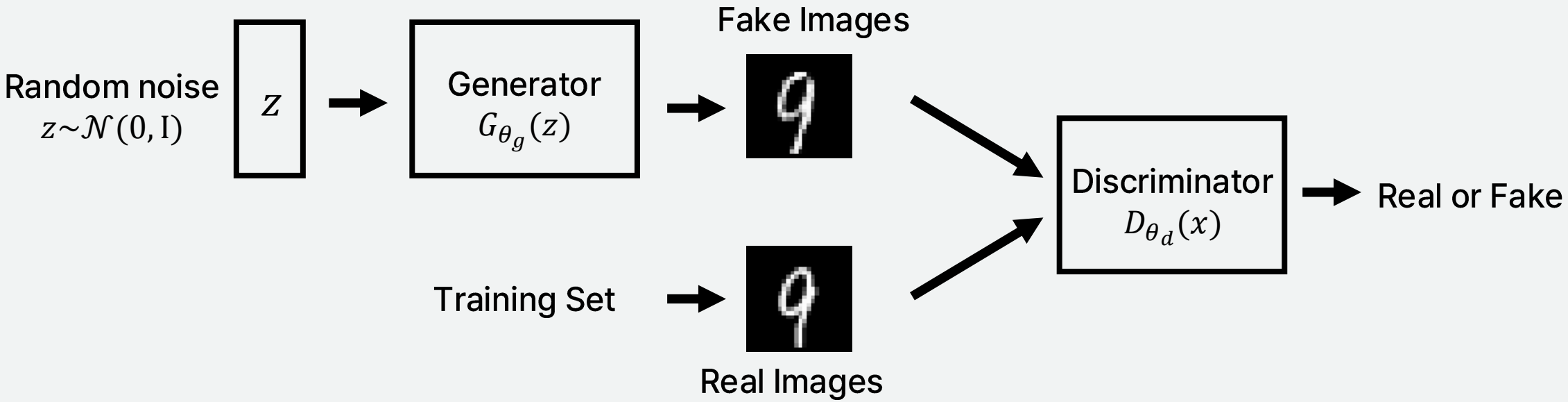


Training GANs



Goodfellow, Ian J., et al. "Generative adversarial nets." NIPS 2014.

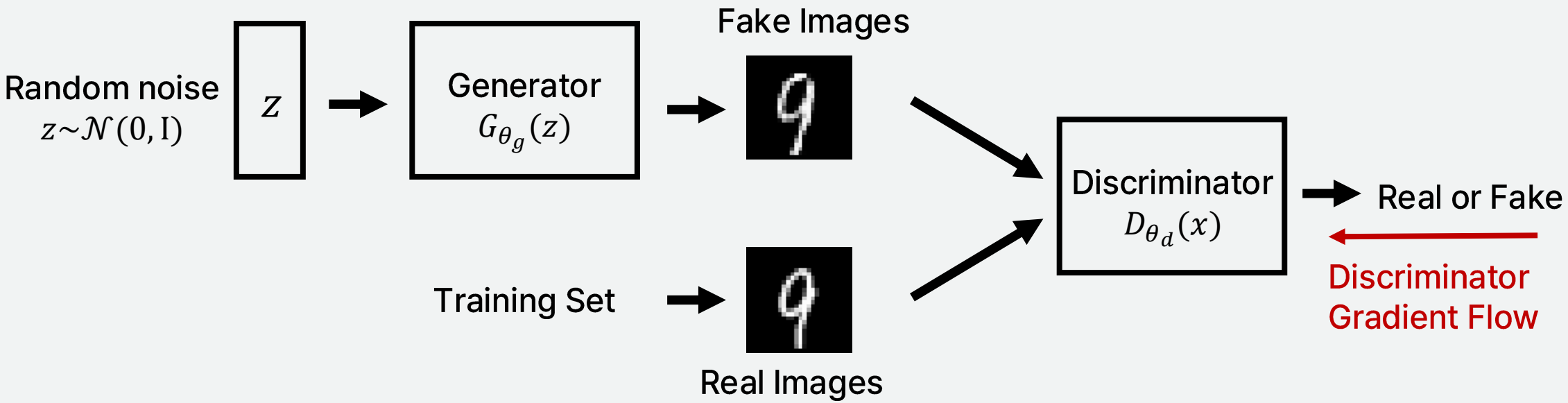
Training GANs



Discriminator Objective : Distinguish fake images from real images

Goodfellow, Ian J., et al. "Generative adversarial nets." NIPS 2014.

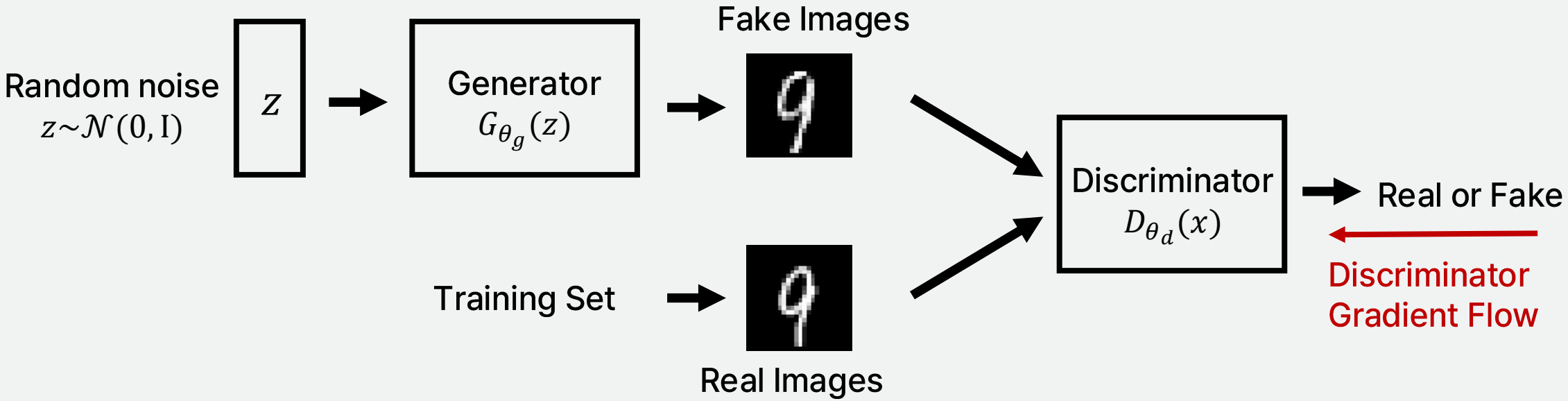
Training GANs



Discriminator Objective : Distinguish fake images from real images

Goodfellow, Ian J., et al. "Generative adversarial nets." NIPS 2014.

Training GANs

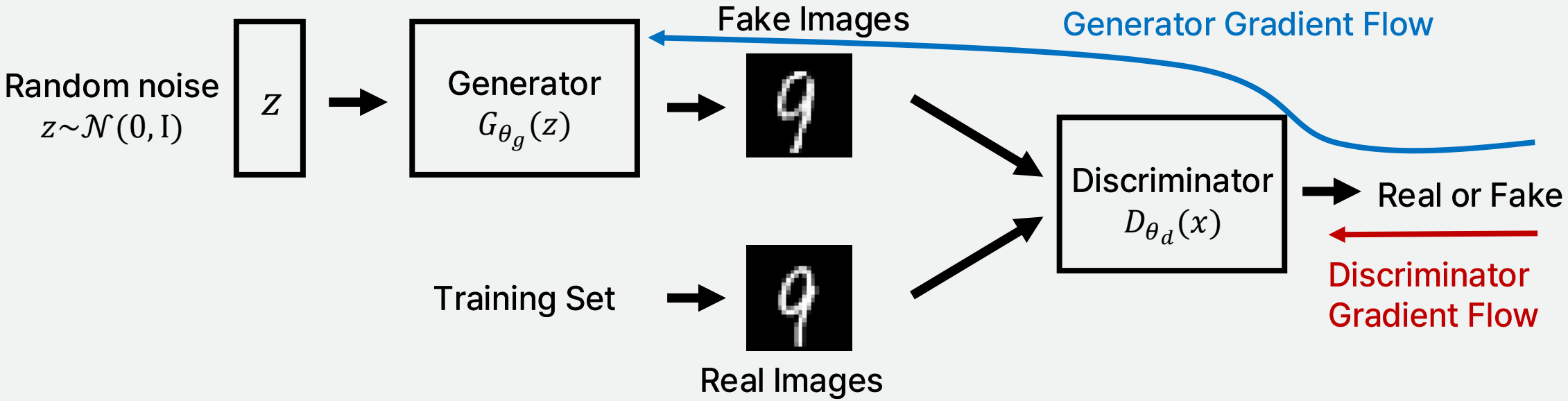


Discriminator Objective : Distinguish fake images from real images

Generator Objective : Fool the discriminator

Goodfellow, Ian J., et al. "Generative adversarial nets." NIPS 2014.

Training GANs

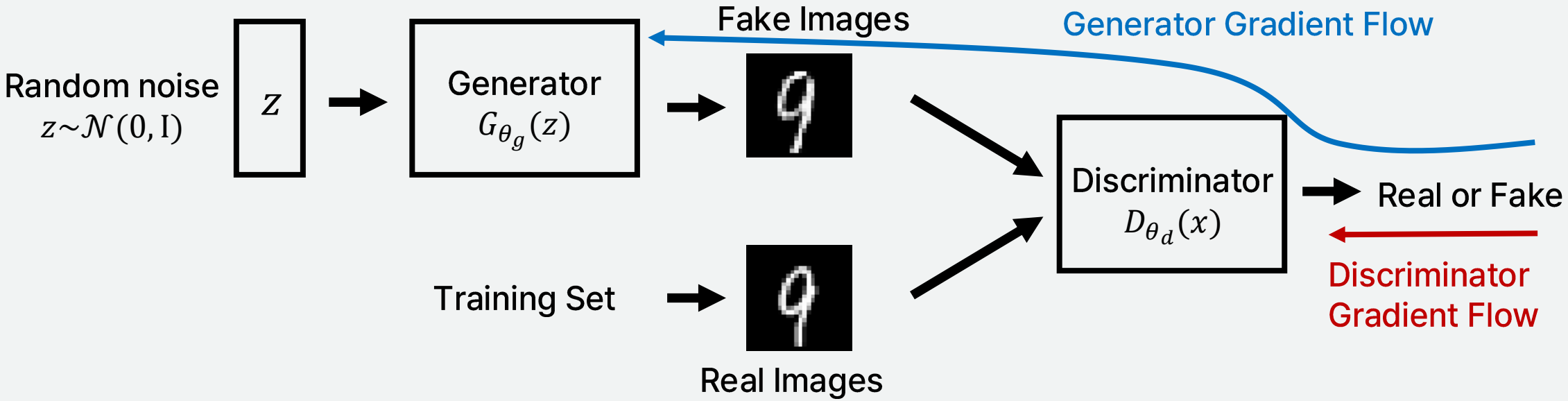


Discriminator Objective : Distinguish fake images from real images

Generator Objective : Fool the discriminator

Goodfellow, Ian J., et al. "Generative adversarial nets." NIPS 2014.

Training GANs



Discriminator Objective : Distinguish fake images from real images

Generator Objective : Fool the discriminator

$$\min_{\theta_g} \max_{\theta_d} \left[\mathbb{E}_{x \sim p_{data}} \log D_{\theta_d}(x) + \mathbb{E}_{z \sim p(z)} \log \left(1 - D_{\theta_d} \left(G_{\theta_g}(z) \right) \right) \right]$$

Goodfellow, Ian J., et al. "Generative adversarial nets." NIPS 2014.

Minimax Game

$$\min_{\theta_g} \max_{\theta_d} \left[\mathbb{E}_{x \sim p_{data}} \log D_{\theta_d}(x) + \mathbb{E}_{z \sim p(z)} \log \left(1 - D_{\theta_d} \left(G_{\theta_g}(z) \right) \right) \right]$$

Goodfellow, Ian J., et al. "Generative adversarial nets." NIPS 2014.

Minimax Game

Discriminator Objective : Distinguish fake images from real images

$$\min_{\theta_g} \max_{\theta_d} \left[\mathbb{E}_{x \sim p_{data}} \log D_{\theta_d}(x) + \mathbb{E}_{z \sim p(z)} \log \left(1 - D_{\theta_d} \left(G_{\theta_g}(z) \right) \right) \right]$$

Goodfellow, Ian J., et al. "Generative adversarial nets." NIPS 2014.

Minimax Game

Discriminator Objective : Distinguish fake images from real images

$$\min_{\theta_g} \max_{\theta_d} \left[\mathbb{E}_{x \sim p_{data}} \log D_{\theta_d}(x) + \mathbb{E}_{z \sim p(z)} \log \left(1 - D_{\theta_d}\left(G_{\theta_g}(z) \right) \right) \right]$$

maximize

minimize

Goodfellow, Ian J., et al. "Generative adversarial nets." NIPS 2014.

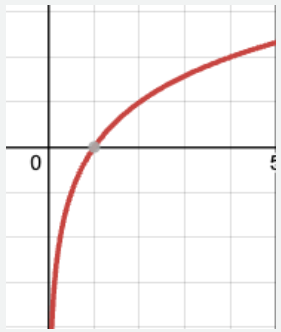
Minimax Game

Discriminator Objective : Distinguish fake images from real images

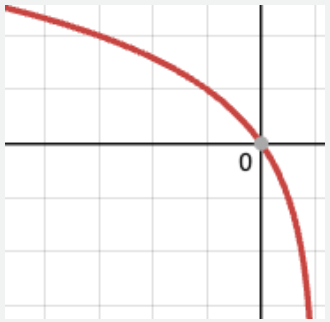
$$\min_{\theta_g} \max_{\theta_d} \left[\mathbb{E}_{x \sim p_{data}} \log D_{\theta_d}(x) + \mathbb{E}_{z \sim p(z)} \log \left(1 - D_{\theta_d}\left(G_{\theta_g}(z) \right) \right) \right]$$

maximize

minimize



log x



log (1-x)

Goodfellow, Ian J., et al. "Generative adversarial nets." NIPS 2014.

Minimax Game

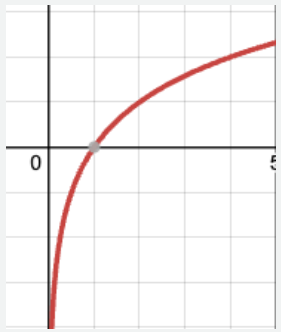
Discriminator Objective : Distinguish fake images from real images

Real data

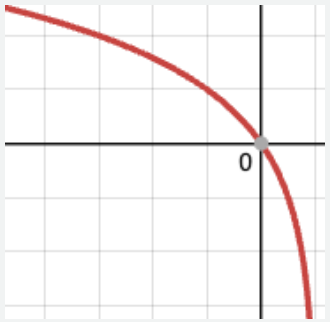
$$\min_{\theta_g} \max_{\theta_d} \left[\mathbb{E}_{x \sim p_{data}} \log D_{\theta_d}(x) + \mathbb{E}_{z \sim p(z)} \log \left(1 - D_{\theta_d}\left(G_{\theta_g}(z) \right) \right) \right]$$

maximize

minimize



log x



log (1-x)

Goodfellow, Ian J., et al. "Generative adversarial nets." NIPS 2014.

Minimax Game

Discriminator Objective : Distinguish fake images from real images

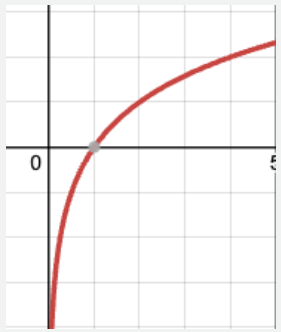
Real data

Fake data

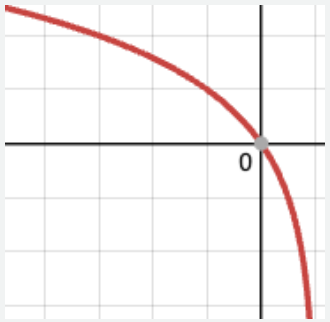
$$\min_{\theta_g} \max_{\theta_d} \left[\mathbb{E}_{x \sim p_{data}} \log D_{\theta_d}(x) + \mathbb{E}_{z \sim p(z)} \log \left(1 - D_{\theta_d}\left(G_{\theta_g}(z) \right) \right) \right]$$

maximize

minimize



log x



log (1-x)

Goodfellow, Ian J., et al. "Generative adversarial nets." NIPS 2014.

Minimax Game

Discriminator Objective : Distinguish fake images from real images
Generator Objective : Fool the discriminator

$$\min_{\theta_g} \max_{\theta_d} \left[\mathbb{E}_{x \sim p_{data}} \log D_{\theta_d}(x) + \mathbb{E}_{z \sim p(z)} \log \left(1 - D_{\theta_d} \left(G_{\theta_g}(z) \right) \right) \right]$$

Goodfellow, Ian J., et al. "Generative adversarial nets." NIPS 2014.

Minimax Game

Discriminator Objective : Distinguish fake images from real images
Generator Objective : Fool the discriminator

Independent of generator

$$\min_{\theta_g} \max_{\theta_d} \left[\mathbb{E}_{x \sim p_{data}} \log D_{\theta_d}(x) + \mathbb{E}_{z \sim p(z)} \log \left(1 - D_{\theta_d} \left(G_{\theta_g}(z) \right) \right) \right]$$

Goodfellow, Ian J., et al. "Generative adversarial nets." NIPS 2014.

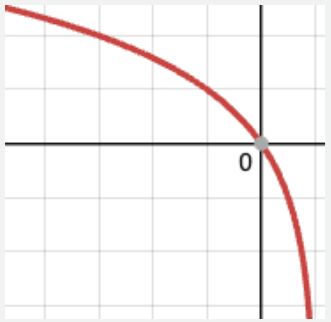
Minimax Game

Discriminator Objective : Distinguish fake images from real images
Generator Objective : Fool the discriminator

Independent of generator

$$\min_{\theta_g} \max_{\theta_d} \left[\mathbb{E}_{x \sim p_{data}} \log D_{\theta_d}(x) + \mathbb{E}_{z \sim p(z)} \log \left(1 - D_{\theta_d}\left(G_{\theta_g}(z) \right) \right) \right]$$

maximize



log (1-x)

Goodfellow, Ian J., et al. "Generative adversarial nets." NIPS 2014.

Minimax Game

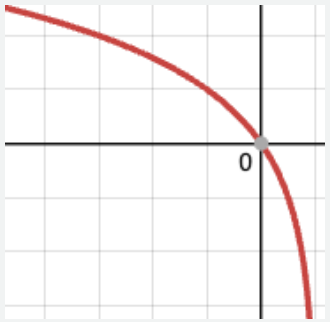
Discriminator Objective : Distinguish fake images from real images
Generator Objective : Fool the discriminator

Independent of generator

Minimize by making D(G(z))->1

$$\min_{\theta_g} \max_{\theta_d} \left[\mathbb{E}_{x \sim p_{data}} \log D_{\theta_d}(x) + \mathbb{E}_{z \sim p(z)} \log \left(1 - D_{\theta_d}\left(G_{\theta_g}(z) \right) \right) \right]$$

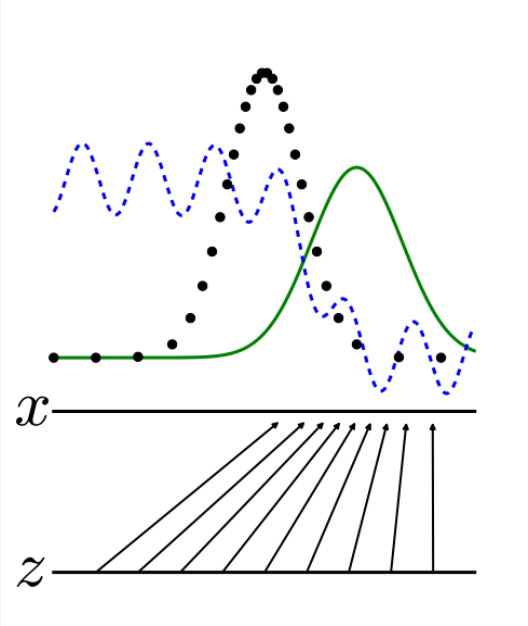
maximize



log (1-x)

Goodfellow, Ian J., et al. "Generative adversarial nets." NIPS 2014.

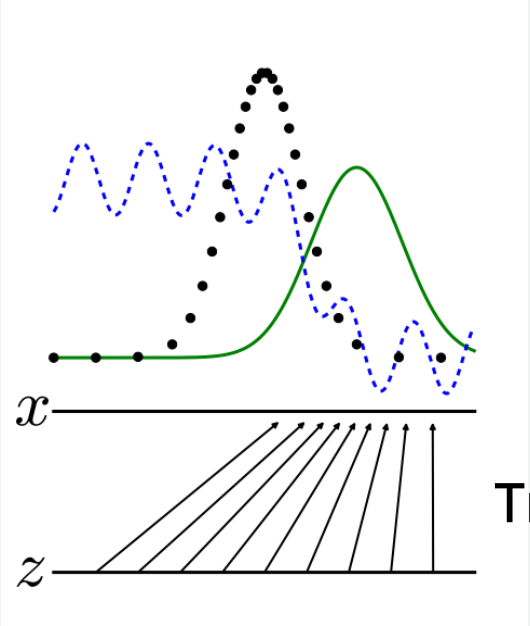
Training Dynamics of GANs



Discriminator : Blue dashed line
Generator : Green solid line
Data distribution : black dotted line

Goodfellow, Ian J., et al. "Generative adversarial nets." NIPS 2014.

Training Dynamics of GANs

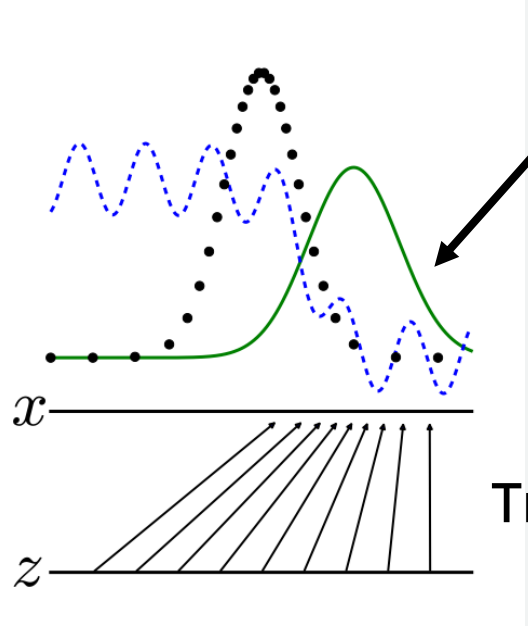


Transformation from distribution p(z)

Discriminator : Blue dashed line
Generator : Green solid line
Data distribution : black dotted line

Goodfellow, Ian J., et al. "Generative adversarial nets." NIPS 2014.

Training Dynamics of GANs



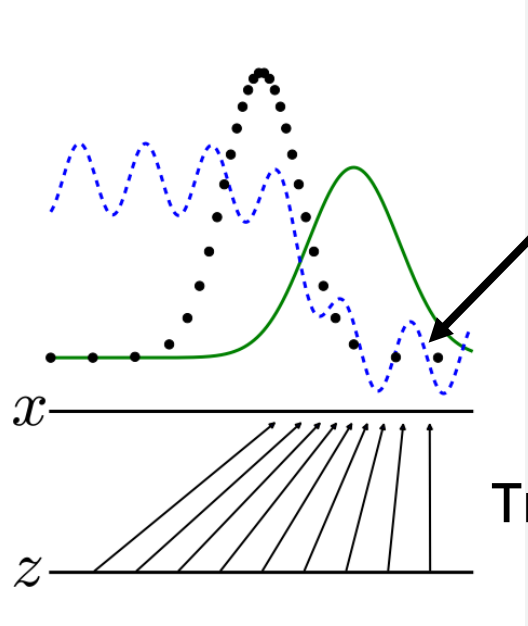
At first, p_g is different from p_{data}

Transformation from distribution p(z)

Discriminator : Blue dashed line
Generator : Green solid line
Data distribution : black dotted line

Goodfellow, Ian J., et al. "Generative adversarial nets." NIPS 2014.

Training Dynamics of GANs



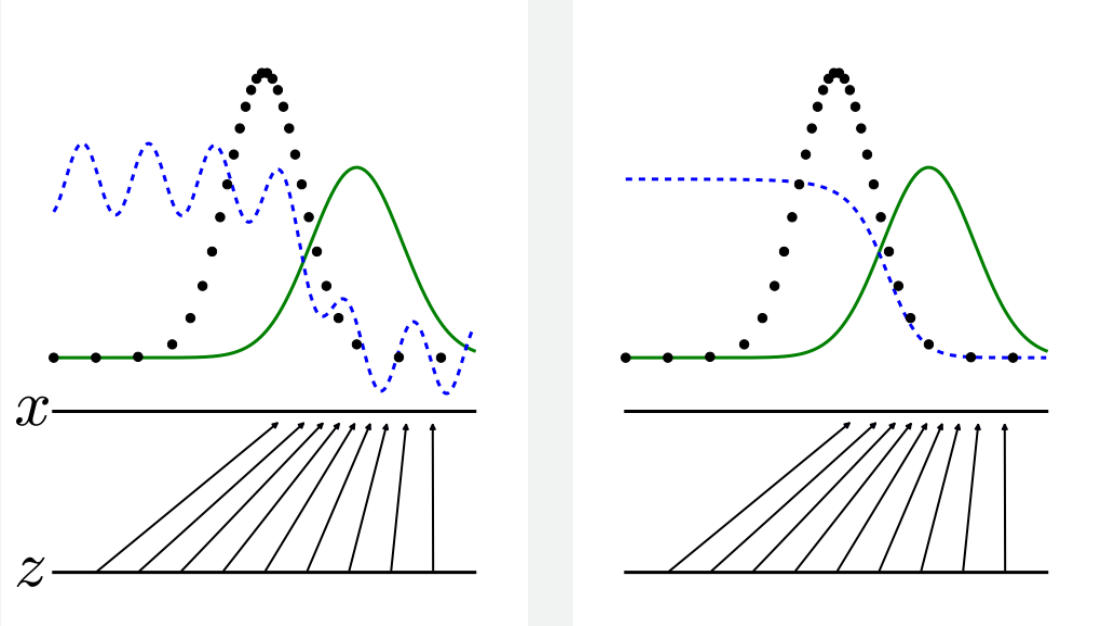
At first, p_g is different from p_{data}
Discriminator can easily distinguish generated samples

Transformation from distribution p(z)

Discriminator : Blue dashed line
Generator : Green solid line
Data distribution : black dotted line

Goodfellow, Ian J., et al. "Generative adversarial nets." NIPS 2014.

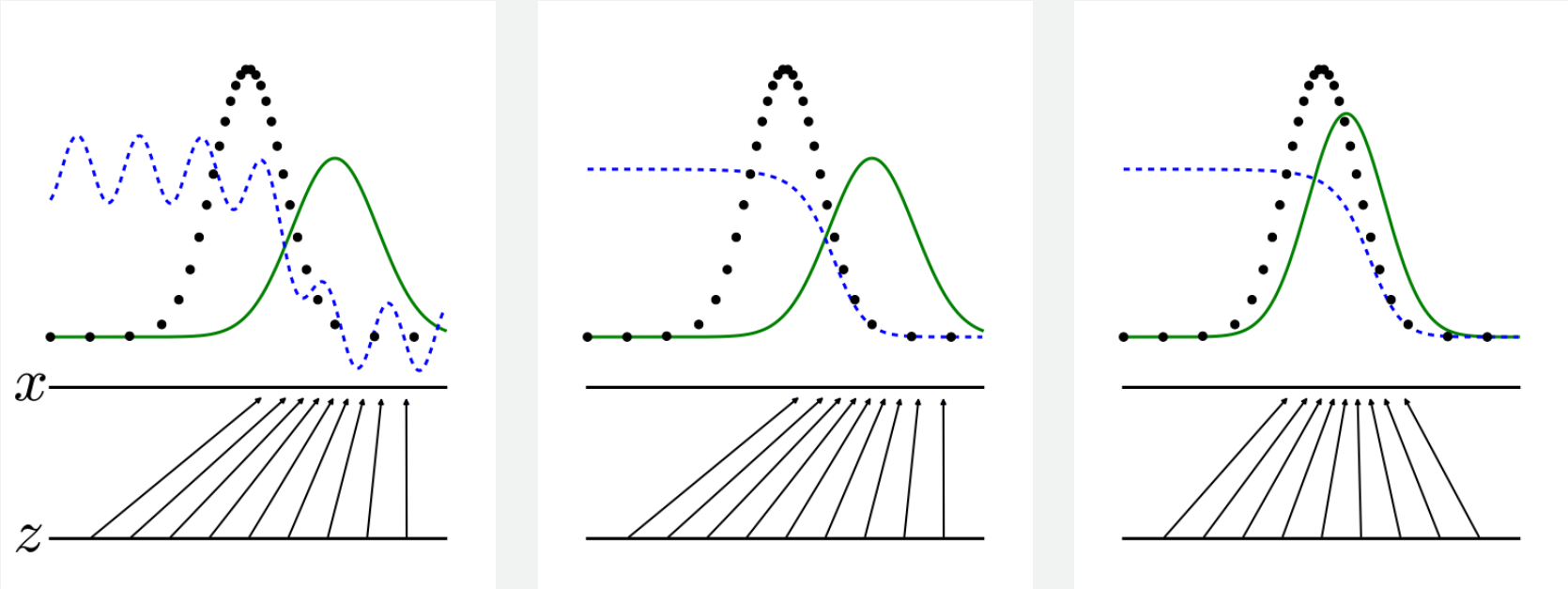
Training Dynamics of GANs



$G(z)$ is trained to map p_g to resemble p_{data}

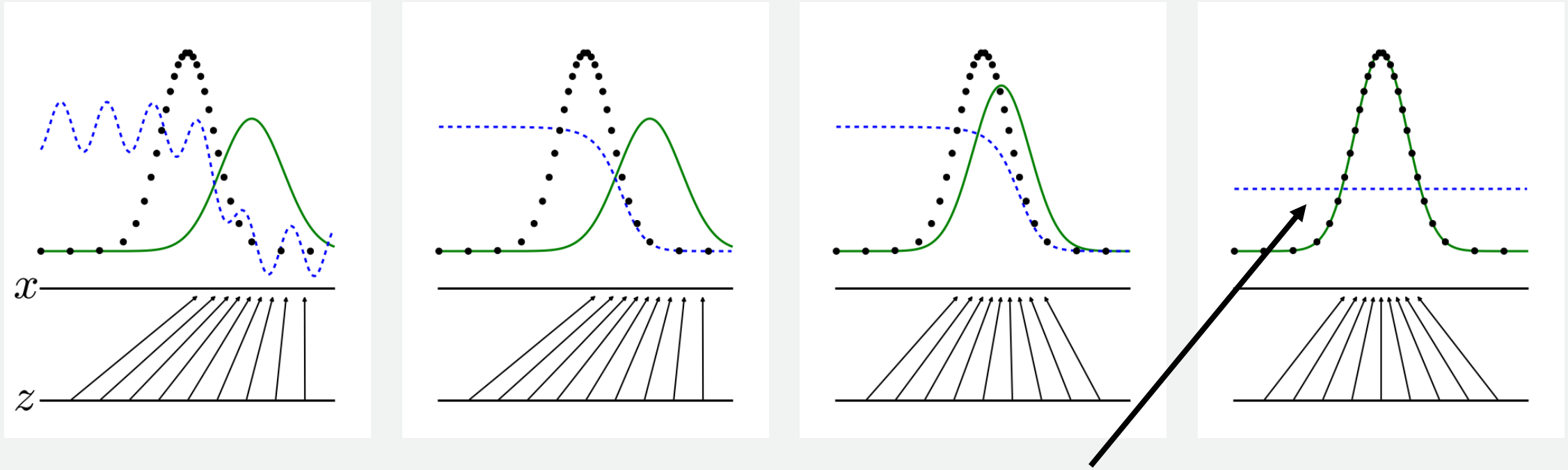
Goodfellow, Ian J., et al. "Generative adversarial nets." NIPS 2014.

Training Dynamics of GANs



Goodfellow, Ian J., et al. "Generative adversarial nets." NIPS 2014.

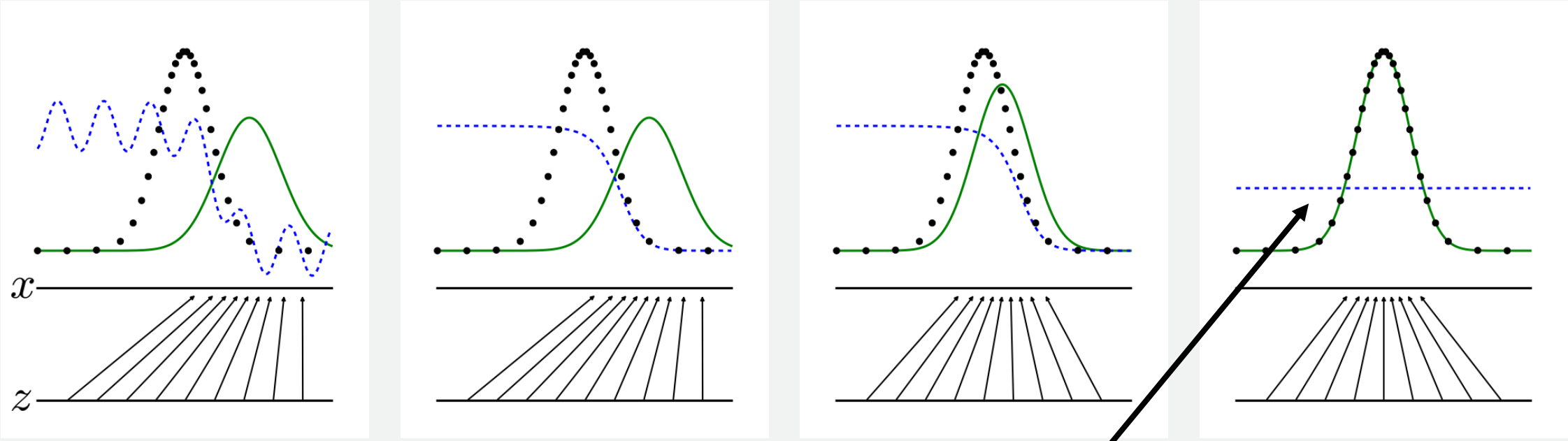
Training Dynamics of GANs



Finally, D cannot discriminate p_g from p_{data},

Goodfellow, Ian J., et al. "Generative adversarial nets." NIPS 2014.

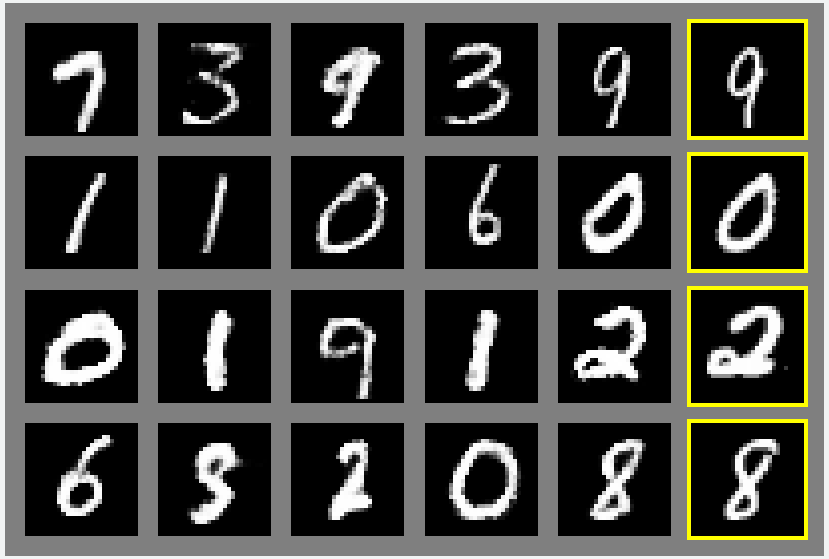
Training Dynamics of GANs



Finally, D cannot discriminate p_g from p_{data}, converging to $D(x) = \frac{p_{data}(x)}{p_{data}(x)+p_g(x)} = \frac{1}{2}$

Goodfellow, Ian J., et al. "Generative adversarial nets." NIPS 2014.

Generated Samples



MNIST Digits Dataset



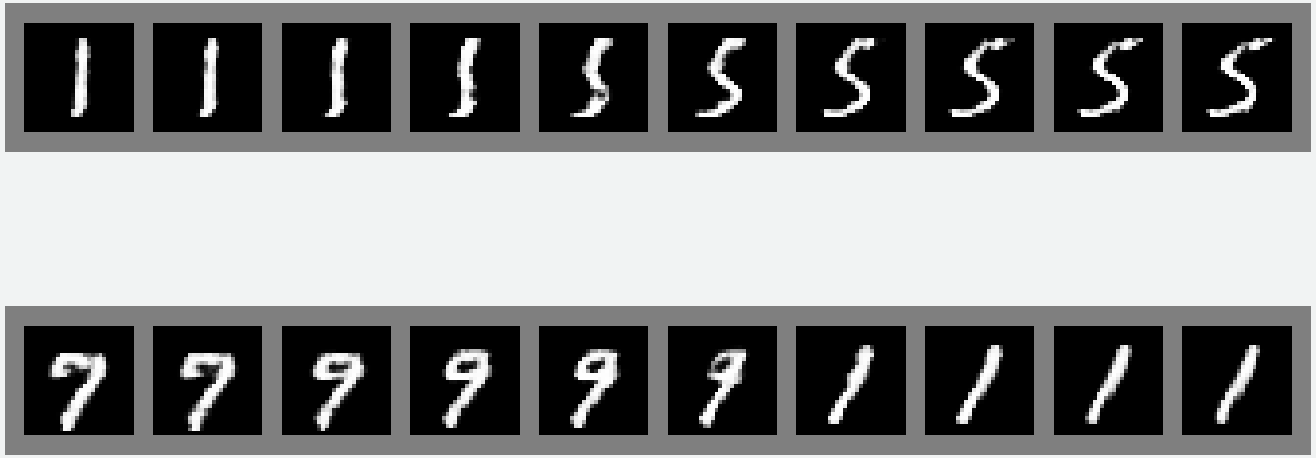
Toronto Faces Dataset

Rightmost column shows the nearest training example

Goodfellow, Ian J., et al. "Generative adversarial nets." NIPS 2014.

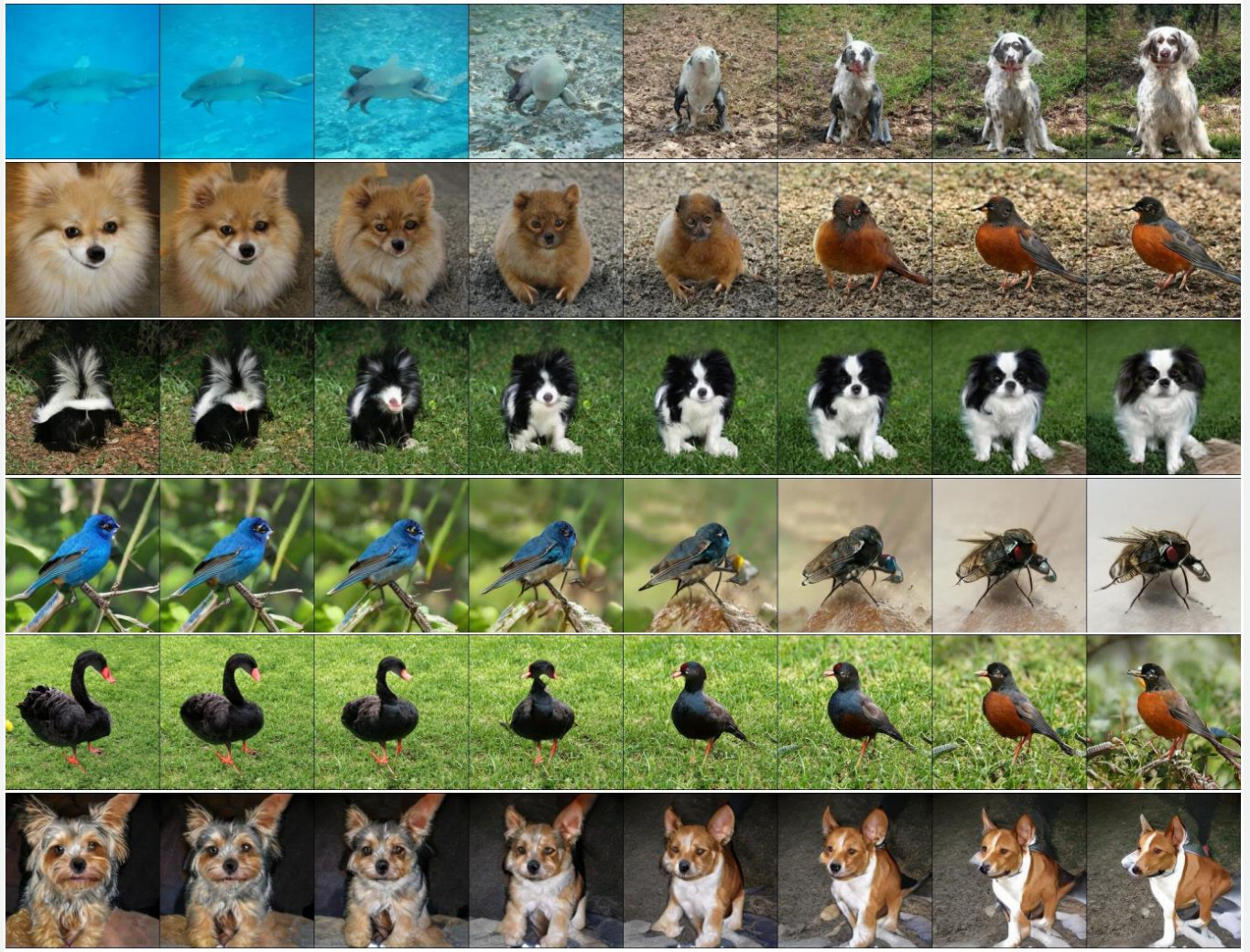
Interpolating Latent Variables

- Interpolating z visualizes transitions in high-level features



Goodfellow, Ian J., et al. "Generative adversarial nets." NIPS 2014.

Interpolating Latent Variables



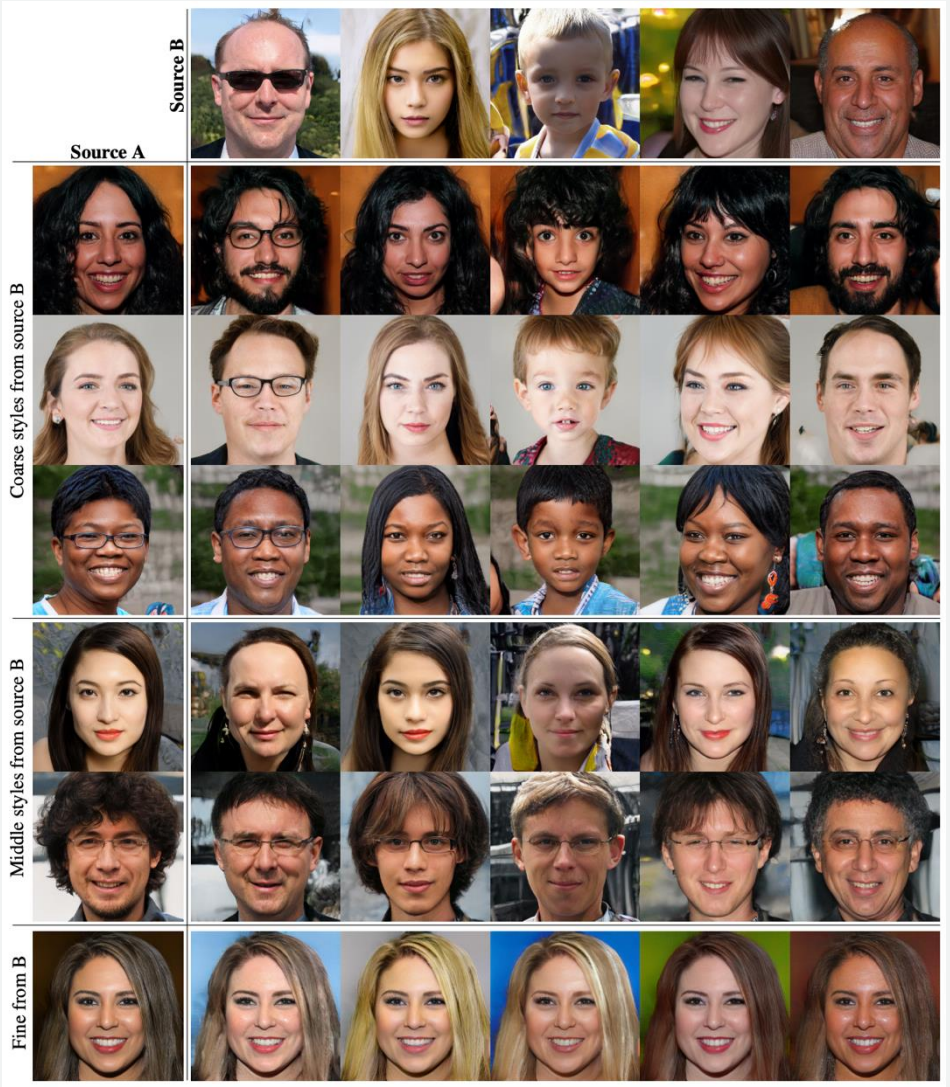
Brock et al., "Large Scale GAN Training For High Fidelity Natural Image Synthesis." ICLR 2019.

High Resolution Image Generation



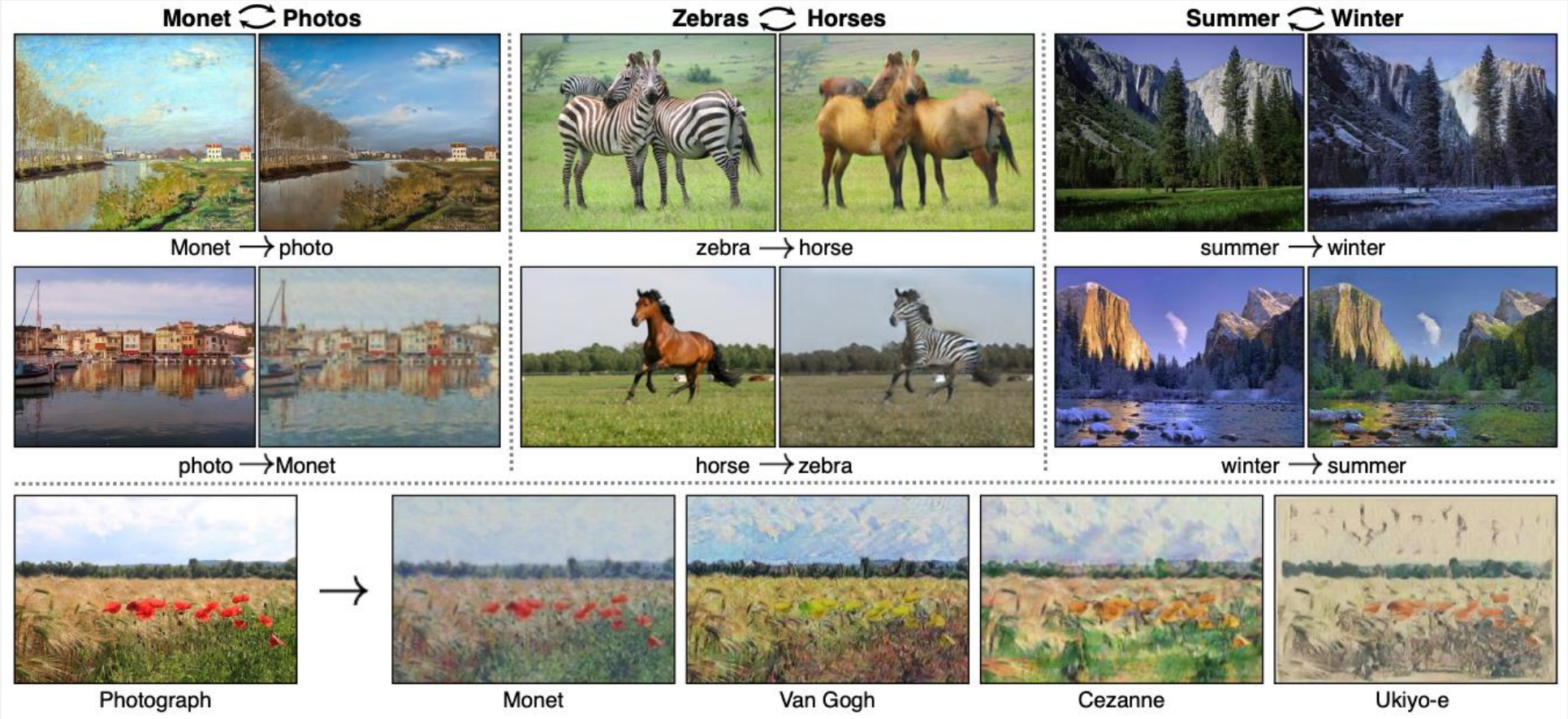
Karrase et al. ”A Style-Based Generator Architecture for Generative Adversarial Networks." CVPR 2019.

Style Transfer



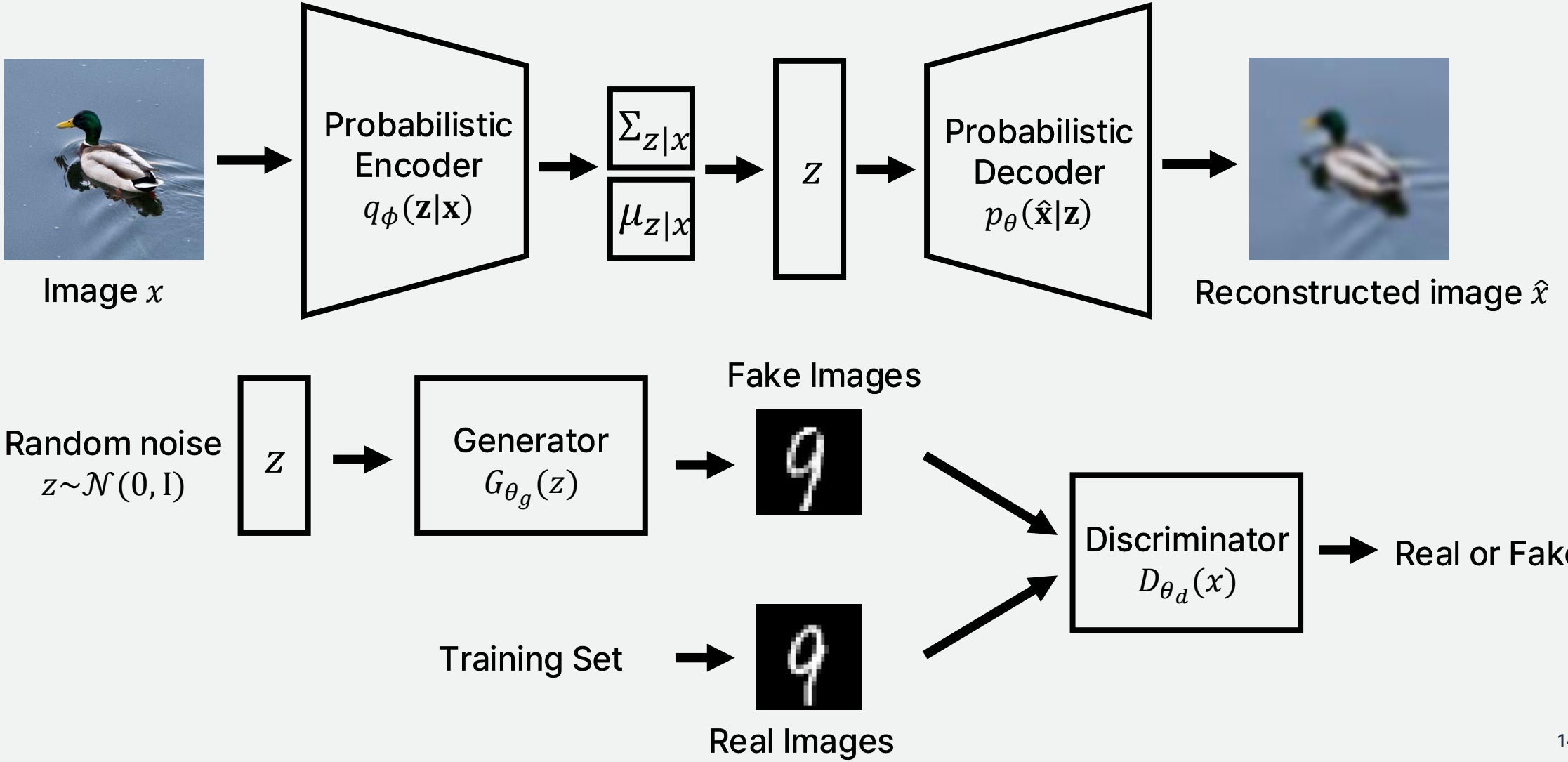
Karrase et al. ”A Style-Based Generator Architecture for Generative Adversarial Networks." CVPR 2019.

Style Transfer



Zhu et al. "Unpaired Image-to-Image Translation using Cycle-Consistent Adversarial Networks." CVPR 2019.

Summary


Image x
Probabilistic Encoder $q_\phi(\mathbf{z}|\mathbf{x})$
$\Sigma_{z|x}$
$\mu_{z|x}$
z
Probabilistic Decoder $p_\theta(\hat{\mathbf{x}}|\mathbf{z})$
Reconstructed image $\hat{x}$
Random noise $z \sim \mathcal{N}(0, \mathrm{I})$
z
Generator $G_{\theta_g}(z)$
Fake Images
Training Set
Real Images
Discriminator $D_{\theta_d}(x)$
Real or Fake

Further Resources

- Stanford cs231n Deep Learning for Computer Vision (Feifei Li et al.)
  - https://cs231n.stanford.edu/
- Stanford cs236 Deep Generative Models (Stefano Ermon et al.)
  - https://deepgenerativemodels.github.io/
- Princeton COS597C Advanced Methods in Probabilistic Modeling, Variational Inference Lecture Note (David M. Blei)
  - https://www.cs.princeton.edu/courses/archive/fall11/cos597C/lectures/variational-inference-i.pdf

Thank you!

Jihwan Hong
AIDAS Lab
Seoul National University

AIDAS Lab